Department of Computer Science and Engineering
PES University, Bangalore, India

UE23CS243A: Automata Formal Language and Logic

Prakash C O, Associate Professor, Department of CSE

Definition of a decidable language.

A language L is called decidable if there exists an algorithm (or, equivalently, a Turing machine) that:

- given a word,
- returns "yes" or "no" depending on whether this word belongs to this language or not.

Decidable language -A decision problem P is said to be decidable (i.e., have an algorithm) if the language L of all yes instances to P is decidable.

Examples

1. (Acceptance problem for DFA) Given a DFA does it accept a given word?
2. (Emptiness problem for DFA) Given a DFA does it accept any word?
3. (Equivalence problem for DFA) Given two DFAs, do they accept the same language?

Note: Not all languages are decidable.

Halting Problem: The Unsolvable Problem

The halting problem can never be solved algorithmically.

Why the Halting Problem Matters in Theory of Computation

- The Halting Problem has an extensive scope and **plays a crucial role in understanding the limitations of computation**.
- It sets the boundary for what computers can and cannot solve and has substantial implications for multiple areas of study, from Artificial Intelligence to Cybersecurity.

Alan Turing and the Halting Problem

- Alan Turing, often referred to as the father of theoretical computer science and artificial intelligence, made significant contributions towards solving the Halting Problem.
- Turing's efforts were instrumental in proving that **a general algorithm that solves the Halting problem for all possible program-input pairs cannot exist.**
  As such, he demonstrated the restrictions of computers, thus establishing a limitation on the power of mechanical computation.

Definitions of Halting Problem

There are many equivalent definitions of Halting Problem:

- Given a TM M and string w, does M halt on w?. Note that M doesn't have to accept w; it just has to halt on w. As a formal language: **$HALT_{TM}$ = { ⟨M, w⟩ | M is a TM and M halts on w. }**

- The Halting Problem, in the simplest terms, is a statement about computational processes in computer science. **Is it possible to write a program that determines whether another program will halt or run indefinitely?**

- **Suppose we have a Turing machine that never halts. Can we make a Turing machine that can detect this? In other words, can we make an infinite loop detector?**

- **Does a TM, say H, exist which takes as its input description of another TM M and a string w and says if M halts on w or not (goes in an infinite loop)?**
  Going by the Church Turing thesis, a computer program/ algorithm is a set of steps that can be executed on a corresponding TM.
- So, this question is same as asking: **can we have a general program/algorithm to solve the halting problem for all possible program-input pairs** (the halting problem is the problem of determining, from a description of an arbitrary computer program and an input, whether the program will finish running, or continue to run forever)?

Undecidability of Languages

1) Prove that Language $HALT_{TM}$ = { <M, w> | M is a TM and M halts on input w } is undecidable

Given a TM M and string w, does M halt on w? Note that M doesn't have to accept w; it just has to halt on w.

Proof: By Self-Reference Method

Assume, for the sake of contradiction, language $HALT_{TM}$ is decidable. This means there exists a **Turing machine H** (i.e., H is a decider and L(H) = $HALT_{TM}$) **that takes as its input < M, w >,** where M is the description of the TM encoded in binary and w is a string, and provides the following output:

H(< M, w >)
- **Accept: if M halts on input w** (M halt at final state / M halt at non-final state on input w)
- **Reject: if M goes into an infinite loop/ doesn't halt upon input w**

Decider H, "On input <M, w>: Run M on w. If M accepts, accept. If M rejects, accept." Then H accepts <M, w> if and only if M halts on w.

If decider H exists, then there exists a TM P such that when < M > (description of a TM M) is applied as an input to P, P calls the decider H and passes on to it an input in the form: **< M, < M >>**. The output of P is 'Accept' if H's output is 'accept'. But P goes in an infinite loop if H's output is 'reject'.

Thus, for the input < M > applied to P, P responds as follows:

P(< M >)
- **Accept:** If M accepts/rejects for input < M >
- **Loop:** If M goes in an infinite loop/doesn't halt for input < M >

The Turing Machine P calls the decider H internally as H(< M, <M> >) or The Turing machine P also simulates the decider H on input < M, < M >>

If decider H exists, also there exists another **TM N which does the reverse of P**, i.e, when < M > is applied as an input to N, N calls the aforementioned decider H and passes on to it an input in the form: < M, < M >>. The output of N is 'accept' if H's output is 'reject'. But N goes in an infinite loop if H's output is 'accept'.
Thus, for the input < M > applied to N, N responds as follows:

N(< M >)
- **Accept:** If M goes in an infinite loop/ doesn't halt for input < M >
- **Loop:** If M accepts/rejects for input < M >

The Turing Machine N calls the decider H internally as H(< M, <M> >) or The Turing machine N also simulates the decider H on input < M, < M >>

Note: While H is a decider, P and N are not deciders (they don't have to be), and so they can go in infinite loops.

What happens when the input to TM N is its own description < N >? i.e., N(< N >)

Now N calls the decider H as H(<N, < N >>)

- **If N goes in an infinite loop/doesn't halt for input < N >, N provides the output Accept** (for the input < N > applied to N). **This is a contradiction.**
- **If N accepts/rejects for input < N >, N goes into an infinite loop** (for the input < N > applied to N). **This is also a contradiction.**

So, for either case, there is a contradiction, and so TM N can't exist. But if decider H exists, TM N must exist.
So decider H can't exist. Hence, our initial assumption that $HALT_{TM}$ is decidable is wrong.
Thus, we prove by contradiction that $HALT_{TM}$ is undecidable.

Proof: By Cantor Diagonalization

Since TMs form a countable set, the following table can be constructed, with each row corresponding to a TM and each column corresponding to a TM description encoded in binary.

Each element (i, j) of the table corresponds to what TM M_i does when description of TM M_j (i.e., $< M_j >$) is fed to the TM M_i as an input string (i= 1, 2, 3, ...; j=1, 2, 3, ...).
There are three possibilities: 1) **accept,** 2) **reject,** 3) **loop (i.e., goes in an infinite loop/doesn't halt)**.
The table is shown below:

Table 1:

Turing Machine	$< M_1 >$	$< M_2 >$	$< M_3 >$	...	...	$< M_N >$	...
M_1	accept	loop	reject	...	...	...	...
M_2	loop	loop	accept	....	...	...	...
M_3	accept	accept	reject	...	...	...	...
...	...	...	...	...	...	...	...
...							
...	...	...	...	...	...	...	...
...							
M_N	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...

If language $HALT_{TM}$ is decidable, it means that there exists a decider H which takes as its input **< M_i, < M_j >>** and yields the following output:

Accept: if M_i halts upon input < M_j > (final state for M_i accept/ reject)
Reject: if M_i goes into an infinite loop/ doesn't halt upon input < M_j >

Corresponding to Table 1, the decider H's table is then as follows

Turing Machine	$< M_1 >$	$< M_2 >$	$< M_3 >$	...	...	$< M_N >$	...
M_1	accept	reject	accept	...	...	...	...
M_2	reject	reject	accept	....	...	...	...
M_3	accept	accept	accept	...	...	...	...
...	...	...	...	...	...	...	...
...							
...	...	...	...	...	...	...	...
...							
M_N	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...

Table 2: A table for the decider H corresponding to $HALT_{TM}$ (assuming $HALT_{TM}$ is decidable)

Using the diagonal in Table 2, we can construct TM P (this is equivalent to P in the self-reference method) which takes as an input < M_i> and acts as follows:

P(< M_i >)
- **Accept:** If M_i accepts/rejects < M_i >
- **Loop:** If M_i goes in an infinite loop/doesn't halt for input < M_i >

The Turing Machine P calls the decider H internally as **H(< M_i, < M_i >>)**or The Turing machine P also simulates the decider H on input **< M_i, < M_i >>**

Loop here means P goes in an infinite loop/ doesn't halt (again, P doesn't need to be a decider).
Thus, from Table 1 and Table 2, **P(< M_1 >)= accept, P(< M_2 >)= loop** (because P is not a decider), **P(< M_3 >)= accept**,.... Hence, it is to be noted that **P is not exactly the diagonal of Table 2 but derived from it ('accept' in the diagonal is 'accept' for P and 'reject' in the diagonal is 'loop' for P).**

We took the diagonal in Table 1 and took its complement, here also **we can do the same for Table 2** and **construct another TM M_N** (By our assumption, H is a TM and so is M_N. Therefore, it must occur on the list M_1, M_2 , . . . of all TMs).

M_N takes as an input < M_i> and does the opposite of P.

Thus, M_N acts as follows:

Accept: If M_i goes in an infinite loop/ doesn't halt for input < M_i >
Loop: If M_i accepts/ rejects on input < M_i >

Thus, from Table 1 and Table 2, **M_N(< M_1 >)= loop, M_N (< M_2 >)= accept, M_N (< M_3 >)= loop,....** Just like complement of the diagonal in Table 1, cannot be placed in Table 1, TM M_N cannot be placed in Table 1.

- If M_N is placed at the N^{th} row of Table 2 as shown below, **TM M_N accepts < M_N >, if M_N goes in an infinite loop upon input < M_N >. This is a contradiction.**
- Similarly, **TM M_N goes in an infinite loop upon input < M_N >, if M_N accepts/rejects input < M_N >. This is also a contradiction.**

Turing Machine	$< M_1 >$	$< M_2 >$	$< M_3 >$	...	...	$< M_N >$	...
M_1	accept	reject	accept	...	...	...	...
M_2	reject	reject	accept	....	...	...	...
M_3	accept	accept	accept	...	...	...	...
...	...	...	...	...	...	...	...
...							
...	...	...	...	...	...	...	...
...							
M_N	reject	accept	reject	...	...	?	...

Table 2: A table for the decider H corresponding to $HALT_{TM}$ (assuming $HALT_{TM}$ is decidable)

But since M_N is a TM, it should correspond to a row in Table-2. But since it can't due to the contradiction, we conclude that decider H can't exist.
Thus, we prove by Cantor's diagonalization technique that decider H doesn't exist and thus $HALT_{TM}$ is undecidable.

Note: No matter what M_N does, it is forced to do the opposite, which is obviously a contradiction. Thus, neither TM M_N nor TM H can exist.

2) Prove that Language A_{TM} = {< M, w > | M is a TM and M accepts input w } is undecidable

The problem of determining whether a Turing machine accepts a given input.

Proof: By Cantor Diagonalization

Assume that a Turing Machine U decides language A_{TM}.

U(< M, w >)
- **Accept:** if M accepts an input w (M halt at final state)
- **Reject:** if M goes into an infinite loop/ halt at non-final state on input w.

We construct a new Turing machine N which takes <M> and calls U internally as U(<M, <M>>).

N(< M >)
- **Accept:** if M goes into an infinite loop/ halt at non-final state on input <M>.
- **Reject:** if M accepts an input <M> (M halt at final state)

Finally, run N on itself, that is, N(<N>) and N calls internally U as U(<N, <N>>)

Thus, the machines take the following actions, with the last line being the contradiction.

- U accepts ⟨M, w⟩ exactly when M accepts w. **i.e., when you call U(<M, w>)**
- N rejects ⟨M⟩ exactly when M accepts input ⟨M⟩. **i.e., when you call N(<M>) and N calls internally U(<M, <M>>)**
- N rejects ⟨N⟩ exactly when N accepts input ⟨N⟩ **i.e., when you call N(<N>) and N calls internally U as U(<N, <N>>)**

We list all TMs down the rows, M1, M2, … , and all their descriptions across the columns, ⟨M1⟩, ⟨M2⟩, … The entries tell whether the machine in a given row accepts the input in a given column.
The entry is *accept* if the machine accepts the input and entry is *blank* if it rejects or loops on that input.

	<M1>	<M2>	<M3>	<M4>	….
M1	accept		accept		….
M2	accept	accept	accept	accept	….
M3					….
M4	accept	accept			….
….	…. ….	…. ….	…. ….	….	….

If we run Turing Machine U on inputs corresponding to above table, U rejects input ⟨M3, ⟨M2 ⟩⟩. U accepts input ⟨M4, ⟨M1 ⟩⟩. Hence, table for U becomes:

	<M1>	<M2>	<M3>	<M4>	….
M1	accept	reject	accept	reject	….
M2	accept	accept	accept	accept	….
M3	reject	reject	reject	reject	….
M4	accept	accept	reject	reject	….
….	…. ….	…. ….	…. ….	….	….

By our assumption, U is a TM and so is N. Therefore, it must occur on the list M1, M2, . . . of all TMs.
Note that N computes the opposite of the diagonal entries.
The contradiction occurs at the point of the question mark where the entry must be the opposite of itself.

	<M1>	<M2>	<M3>	<M4>	….	<N>	….
M1	accept	reject	accept	reject	….	accept	….
M2	accept	accept	accept	accept	….	accept	….
M3	reject	reject	reject	reject	….	reject	….
M4	accept	accept	reject	reject	….	accept	….
….	….	….	….	….	….	….	….
N	reject	reject	accept	accept	….	???	….
….	….	….	….	….	….	….	….

No matter what N does, it is forced to do the opposite, which is obviously a contradiction. Thus, neither TM N nor TM U can exist.

Exercise:

1. Prove that $L^{\Phi}_{TM} = \{<M> | M \text{ is a TM and } L(M) = \Phi\}$ is undecidable

2. Prove that $L^{EQ}_{TM} = \{ <M_1, M_2> | M \text{ are TMs and } L(M_1) = L(M_2)\}$ is undecidable

Is everything about Turing machines undecidable?

1. **L={<M> | L(M) is regular}**
2. **L={<M> | L(M) is context-free}**
3. **L={<M> | L(M) is finite}**
4. **L={<M> | M has 5 states}**
5. **L={<M> | L(M) is a language}**
6. **L={<M> | M makes at least 5 moves on some input}**

Reducibility (or Reductions)

- We will explore now the phenomenon of undecidability outside the realm of problems concerning automata and Turing machines, i.e., we will look at a Post Correspondence Problem (PCP). We will ultimately show its connection with Turing machines and show that solving PCP is equivalent to deciding A_{TM}.
- Since we already know that A_{TM} is undecidable, by equivalence/reduction, we show here that PCP is non-computable for some cases.

What is Reduction (or Reducibility)?

- **A reduction is a process that converts one problem into another so that the solution of the second problem can be used to solve the first. We say the first problem reduces to the second problem.**
  In other words, a reduction is a way to change a problem that needs to be solved to a problem that is already known how to solve.
- **Reducibility always involves two problems, which we call A and B. If A reduces to B, we can use a solution of B to solve A.**
- Reducibility plays an important role in classifying problems by decidability, and later in complexity theory as well. **When A is reducible to B, solving A cannot be harder than solving B because a solution to B gives a solution to A.**

In terms of computability theory,

- **If A reduces to B and B is decidable, A also is decidable.**
- **Equivalently, if A reduces to B and A is undecidable, then B is also undecidable.**

Note: We say A is reducible to B, and we denote it by $A \leq_m B$. (Mapping reduction)

Definition of the PCP Problem:

Let Σ be an alphabet with at least two symbols. The input of the problem consists of 2-finite lists $x_1, x_2, \ldots, x_n$ and $y_1, y_2, \ldots, y_n$ of n-words over Σ.

A solution to this problem is a sequence of indices, such that the concatenation of the words from the respective list in that sequence yields the same resultant string.

The decision problem then is to decide whether such a solution exists or not, for the given two lists above. We will prove here that this problem is unsolvable by any algorithm.

We can express the PCP as a language as:

L_{PCP} = {< P >|P is an instance of the PCP with a solution/match}

Example 1: Proving Undecidability of L_{PCP} by reducing PCP to empty Intersection problem (i.e., $L_1 \cap L_2 = \emptyset$?)

Solution:

Take a PCP problem, and then write two grammars (corresponding to two lists of PCP).

Consider a PCP Problem

	List-A	List-B
Word-1	1	111
Word-2	10111	10
Word-3	10	0

Write grammars i.e., CFGs for both the List-A and List-B.

CFG G_A for List-A: $S_A \rightarrow 1\ S_A\ 1 \mid 10111\ S_A\ 2 \mid 10\ S_A\ 3 \mid -$

CFG G_B for List-B: $S_B \rightarrow 111\ S_B\ 1 \mid 10\ S_B\ 2 \mid 0\ S_B\ 3 \mid -$

Numbers 1, 2 and 3 after S_A or S_B keeps track of the sequence numbers (way to remember sequence numbers).

We know this PCP has a solution: **2 1 1 3**

Let's generate the string **101111110** using $S_A \rightarrow 1\ S_A\ 1 \mid 10111\ S_A\ 2 \mid 10\ S_A\ 3 \mid -$

$S_A \Rightarrow 10111\mathbf{S_A}2$
$\Rightarrow 101111\mathbf{S_A}12$
$\Rightarrow 1011111\mathbf{S_A}112$
$\Rightarrow 101111110\mathbf{S_A}3112$
$\Rightarrow$ 101111110-3112 (reverse 3112 as 2113 because sequence numbers 1, 2 and 3 are nested in RHS of rules)
$\Rightarrow$ **101111110-2113**

Let's generate the string **101111110** using $S_B \rightarrow 111\ S_B\ 1 \mid 10\ S_B\ 2 \mid 0\ S_B\ 3 \mid -$

$S_B \Rightarrow 10\mathbf{S_B}2$
$\Rightarrow 10\mathbf{111}\mathbf{S_B}12$
$\Rightarrow 10111\mathbf{111}\mathbf{S_B}112$
$\Rightarrow 1011111\mathbf{0}\mathbf{S_B}3112$
$\Rightarrow$ 101111110-3112 (reverse 3112 as 2113 because sequence numbers 1, 2 and 3 are nested in RHS of rules)
$\Rightarrow$ **101111110-2113**

The above derivations of both G_A and G_B are generating the same string and hence the constructed grammars for List-A and List-B are correct.

Grammars G_A and G_B are recursive grammars and represent infinite set context free languages **L(G_A)** and **L(G_B)**. Hence, PCP is reduced to empty intersection problem i.e., **L(G_A)∩L(G_B)=∅?**

We know that, the empty intersection of two CFLs is undecidable, i.e., L1∩L2=∅?

As there is no algorithmic way to solve $L(G_A) \cap L(G_B) = \emptyset$?, hence L_{PCP} is also undecidable(not solvable).

In general PCP is known to be Undecidable, that is, there is no particular algorithm that determines whether any PCP has solution or not.

Exercises:

1. ### Proving Undecidability of PCP by reducing A_{TM} to PCP

   We have already seen that A_{TM} is an undecidable language. In order to prove that PCP is undecidable, it suffices to show that we can reduce A_{TM} to PCP. Thus, we will show that if we can decide PCP, then we can decide A_{TM}. Given any Turing machine M and input w, we construct an instance $P_{M,w}$ such that $P_{M,w}$ has a solution index sequence (or match) if M accepts w. That is, $P_{M,w}$ has a match if there is an accepting CH of M on w. So, if we can determine whether the instance has a match, we would be able to determine whether M accepts w.

   ……

2. ### Proving Undecidability of empty Intersection problem by reducing it to PCP

References:

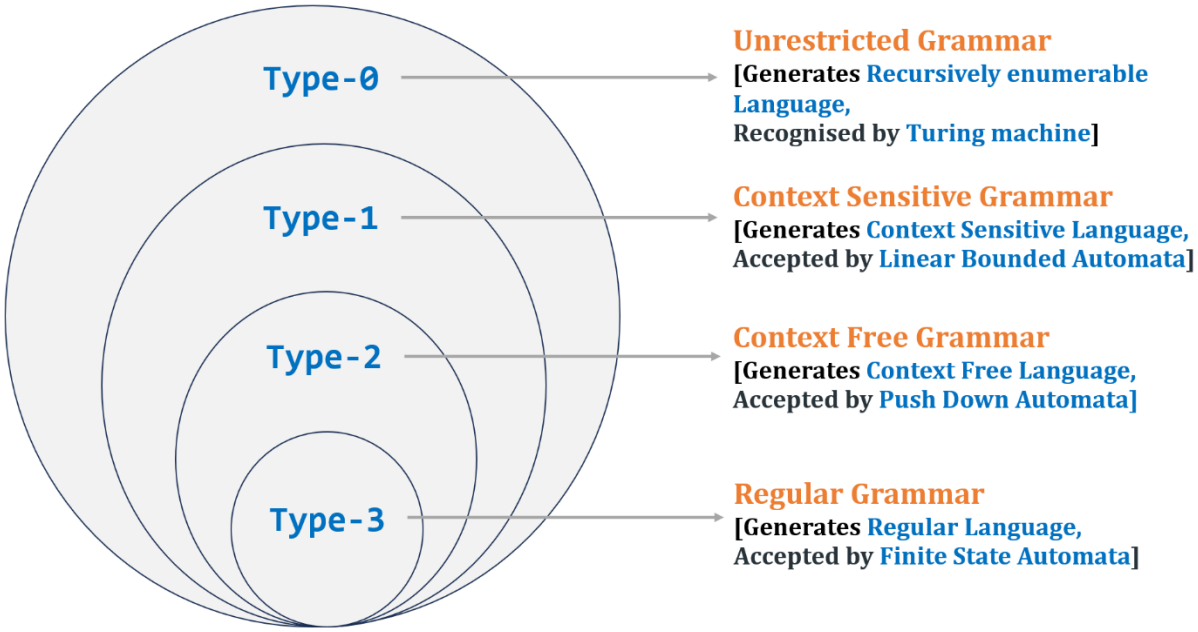
- https://web.iitd.ac.in/~debanjan/
- Undecidable(PCP,UTM,Reduction).pdf by Prof. Preet Kanwal, PES University.

Department of Computer Science and Engineering
PES University, Bangalore, India

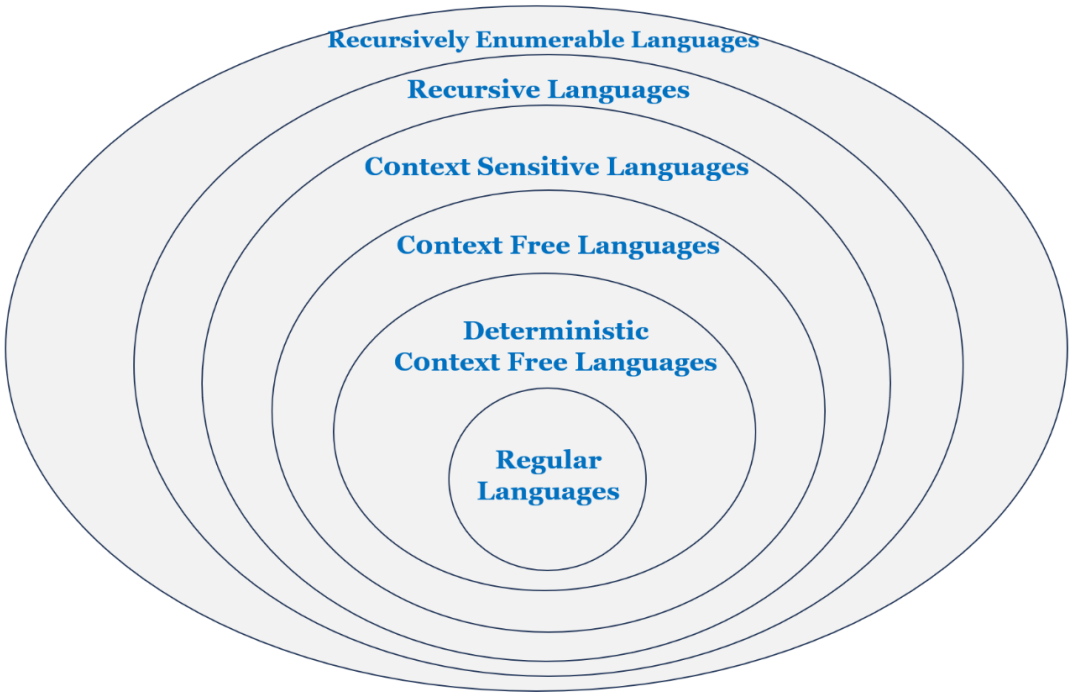
UE23CS243A: Automata Formal Language and Logic

Prakash C O, Associate Professor, Department of CSE

Chomsky hierarchy



All languages over Σ



Note:

1) Type-0 grammar (or unrestricted Grammar)

- Unrestricted grammars are the most general type of grammar in the Chomsky hierarchy. They are also known as phrase structure grammars.
- A grammar G = (V, T, P, S) is called unrestricted if all the productions are of the form **u→v**, where $u \in (V \cup T)^+$ and $v \in (V \cup T)^*$.
- **No restrictions are made on the production rules of an unrestricted grammar**, other than each of their left-hand sides being non-empty.
- **Any language generated by an unrestricted grammar is recursively enumerable.**
- The unrestricted grammars characterize the recursively enumerable languages. This is the same as saying that for every unrestricted grammar G there exists some Turing machine capable of recognizing L(G) and vice versa.

2) Decidable language

A language L is called decidable if there exists an algorithm (or, equivalently, a Turing machine) that:

- given an input string,
- returns "yes" or "no" depending on whether this word belongs to this language or not.

3) Recursively Enumerable Set (Countable set)

In computability theory, **a set S is called countable**, computably enumerable(CE), **recursively enumerable(RE)**, semi-decidable, partially decidable, listable, provable or **Turing-recognizable if**:

- **There is a procedure that enumerates the members of set S**. That means that its output is simply a list of all the members of S: s_1, s_2, s_3, ... If S is infinite, this procedure will run forever.
- **If its elements can be put into a one-to-one correspondence with the natural numbers**. By this we mean that the elements of the set can be written in some order, say, x_1, x_2, x_3, ..., **so that every element of the set has some finite index**.
- If it has a bijection (one-to-one correspondence i.e., we can index each element) with the natural numbers, and is computably enumerable (CE) if there exists a procedure that enumerates its members. Any non-finite computably enumerable set must be countable since we can construct a bijection from the enumeration.
- **We can generate any element** (irrespective of how large the element is) **of the set S in finite amount of time**.

Not every set is countable. There are some uncountable sets. But **any set for which an enumeration procedure exists is countable because the enumeration gives the required sequence**. Strictly speaking, an enumeration procedure cannot be called an algorithm since it will not terminate when S is infinite.

Examples of Recursively Enumerable (or Countable) sets:

1. **Set of odd/even numbers**
2. **Set of prime numbers**
3. **Σ* - Set of all strings over the input alphabet Σ**

   Answer: If Σ={a,b}, How should we order Σ* to show that it is countable?
   We cannot use the sequence λ, `a, aa, aaa, aaaa, ...`

because then b's strings would never appear. This does not imply that the set is uncountable; in this case, there is a clever way of ordering the set to show that it is in fact countable.
Write down strings in the ascending order of string length and among strings of equal length, order them alphabetically.

λ, a, b, aa, ab, ba, bb, aaa, aab, aba, abb, …

Here the string ba occurs in the sixth place, and every element has some position in the sequence.

The set is therefore countable.

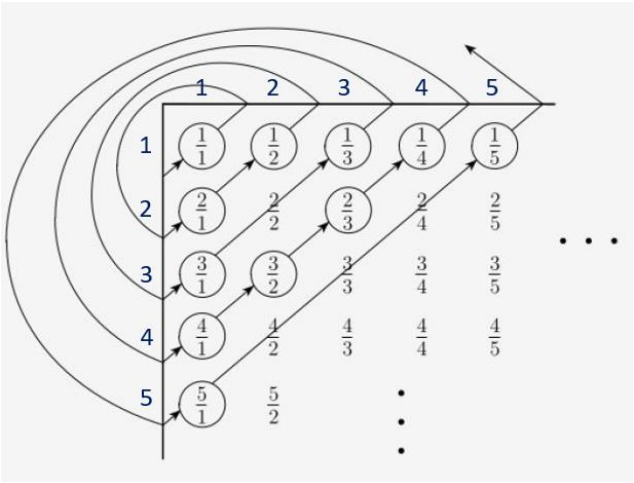
4. The set of all Turing machines (encoded in binary)

Answer: We know that, we can encode the Transition table of a Turing Machine as Description i.e., a unique binary string of finite length. Encode all Turing machines (**i.e., every Turing machine is a binary string of finite length)**.
The total number of possible Turing Machines (i.e., Descriptions encoded in binary) is less than the size of Σ^* for the binary alphabet $\Sigma = \{0,1\}$.

Write down the encoded binary strings in the ascending order of string length and among strings of equal length, order them alphabetically. Hence every encoded string has some position in the sequence. The set is therefore countable.
There are countably infinite numbers of Turing Machines in the world.

5. Set of positive rational numbers i.e., S = { p/q | p, q ∈ ℕ }

Answer: How should we order this set to show that it is countable?
We cannot use the sequence `1/1, 1/2, 1/3, 1/4, …`
because then `2/3` would never appear. This does not imply that the set is uncountable; in this case, there is a clever way of ordering the set to show that it is in fact countable.



Row headings indicates numerator and Column headings indicates denominator

Look at the scheme depicted in Figure above, and write down the element in the order encountered following the arrows.
This gives us 1/1, 2/1, 1/2, 3/1, 1/3, … (enumeration procedure eliminates duplicates such as 2/2, 3/3, 4/2, 2/4, ….)
Here the element 1/3 occurs in the fifth place, and **every element has some position in the sequence**.
The set is therefore countable.

Note: The above set of rational numbers is infinite and have the same size (cardinality) as that of ℕ as we are able to keep the elements from both the sets in one-to-one correspondence.

Cantor proposed this idea(diagonalization) to enumerate the elements of infinite sets.
Cantor's idea of counting the elements, pair with the elements of the other sets.
A set is countable if either it is finite or it has same size as $\mathbb{N}$.

4) So are all infinite sets countable?

Georg Cantor showed that not all infinite sets are countably infinite.

Non-Recursively Enumerable Sets

Cantor's diagonalization proof, was published in 1891 by Georg Cantor as a mathematical proof that there are infinite sets which cannot be put into one-to-one correspondence with the infinite set of natural numbers. Such sets are now known as **uncountable sets**.

1) Real numbers are not countable.

Cantor shocked the world by showing that the real numbers are not countable… there are "more" of them than the integers! His proof was an ingenious use of a proof by contradiction.

In fact, he could show that there exist infinities of many different "sizes"!

Solution 1:

Georg Cantor proved this astonishing fact in 1895 by showing that the set of real numbers is not countable. That is, **it is impossible to construct a bijection between $\mathbb{N}$ and R**. In fact, it's impossible to construct a bijection between $\mathbb{N}$ and the interval [0, 1] (whose cardinality is the same as that of R).

Here's Cantor's proof.

Suppose that $\boldsymbol{f: \mathbb{N} \to [0, 1]}$ is any function. Make a table of values of *f*, where the 1st row contains the decimal expansion of *f(1)*, the 2nd row contains the decimal expansion of *f(2)*, . . . the nth row contains the decimal expansion of *f(n)*, . . . Perhaps f(1) = π/10, f(2) = 37/99, f(3) = 1/7, f(4) = √2/2, f(5) = 3/8, so that the table starts out like this.

n	$f(n)$											
1	0	.	3	1	4	1	5	9	2	6	5	3	. . .
2	0	.	3	7	3	7	3	7	3	7	3	7	. . .
3	0	.	1	4	2	8	5	7	1	4	2	8	. . .
4	0	.	7	0	7	1	0	6	7	8	1	1	. . .
5	0	.	3	7	5	0	0	0	0	0	0	0	. . .
⋮	⋮												

Of course, only part of the table can be shown — it goes on forever down and to the right. Can f possibly be onto? That is, can every number in [0, 1] appear somewhere in the table? In fact, the answer is no — there are lots and lots of numbers that can't possibly appear! For example, **let's highlight the digits in the main diagonal of the decimal part in table.**

n	$f(n)$											
1	0	.	**3**	1	4	1	5	9	2	6	5	3	. . .
2	0	.	3	**7**	3	7	3	7	3	7	3	7	. . .
3	0	.	1	4	**2**	8	5	7	1	4	2	8	. . .
4	0	.	7	0	7	**1**	0	6	7	8	1	1	. . .
5	0	.	3	7	5	0	**0**	0	0	0	0	0	. . .
⋮	⋮												

The highlighted digits are `0.37210....` Suppose that we add 1 to each of these digits, to get the number

`0.48321....`

Now, this number can't be in the table. Why not? Because

- it differs from f(1) in its first digit;
- it differs from f(2) in its second digit;
- . . .
- it differs from f(n) in its nth digit;
- . . .

So it can't equal f(n) for any n — **that is, it can't appear in the table.**

This looks like a trick, but in fact there are lots of numbers that are not in the table. For example,

- we could subtract 1 from each of the highlighted digits (changing 0's to 9's), getting 0.26109 — by the same argument, this number isn't in the table. Or
- we could subtract 3 from the odd-numbered digits and add 4 to the even-numbered digits. Or
- we could even highlight a different set of digits:

n	$f(n)$											
1	0	.	3	**1**	4	1	5	9	2	6	5	3	. . .
2	0	.	**3**	7	3	7	3	7	3	7	3	7	. . .
3	0	.	1	4	2	**8**	5	7	1	4	2	8	. . .
4	0	.	7	0	**7**	1	0	6	7	8	1	1	. . .
5	0	.	3	7	5	0	**0**	0	0	0	0	0	. . .
⋮	⋮												

As long as we highlight at least one digit in each row and at most one digit in each column, we can change each the digits to get another number not in the table. Here, if we add 1 to all the highlighted digits, we end up with 0.42981 . . . — there's a real number that does not equal f(n) for any positive integer n.

What is the point of all this? Precisely that the function f can't possibly be onto — there will always be (infinitely many!) missing values. Therefore, there does not exist a bijection between N and real numbers between 0 and 1.

Solution 2:

Cantor's diagonalization argument goes as follows. If the real numbers were countable, it can be put in 1-1 correspondence with the natural numbers, so we can list them in the order given by those natural numbers.

Now use this list to construct a real number X that differs from every number in our list in at least one decimal place, by letting X differ from the i^{th} digit in the i^{th} decimal place. (A little care must be exercised to ensure that X does not contain an infinite string of 9's.) This gives a contradiction, because the list

was supposed to contain all real numbers, including X. Therefore, hence a 1-1 correspondence of the reals with the natural numbers must not be possible.

2) Set of all languages are not enumerable

(i.e., For any non-empty Σ, there exist languages that are not recursively enumerable)

Solution:

Any formal language L is a subset of **Σ*, L ⊆ Σ*,** we know that Σ* is enumerable (countably infinite)
However, a set of formal languages is not enumerable (uncountably infinite). Since any language is a subset of **Σ*.**
Total number of subsets (or languages) that can be formed using the set Σ* is 2^{Σ^*}.

Formal Proof:

Set of all languages over Σ are enumerable. This means the languages can be numbered as L1, L2, L3, …
Let's construct a 2-D matrix with an infinite number of rows.
Rows are enumerated as languages from the power set $\mathbf{2}^{\Sigma^*}$. We can also see the rows as enumeration of Turing Machines with row Li corresponding to the language of the i^{th} Turing Machine.
Columns in the matrix are all the strings from **Σ* = {s1, s2, s3, …}**

	s1	s2	s3	s4	s5	….
L1	1	0	1	0	1	….
L2	1	1	1	0	1	….
L3	1	0	0	1	1	….
L4	1	1	1	1	1	….
L5	1	0	0	0	0	….
….	….	….	….	….	….	….

Row headings indicates Languages and Column headings indicates strings. Cell value 1 indicates string belongs to the language, and o indicates string does not belong to the language.

Consider left to right diagonal, this will represent another language lets say L_{diag}.
Thus, L_{diag} = { s1, s2, s4, …}

This language L_{diag} must be one of the enumerated languages (must be same as one of the rows)
Thus, the language is computable and recursively enumerable since we can construct Turing machine for such a language (as per our assumption)
Complement this language, what we get is a new language L^c_{diag} = { s3, s5, … }
This new language is not the same as any of the enumerated elements (rows)because it differs from i^{th} diagonal place. Hence it is a new language and not the same as any of the enumerated languages.
This is contradiction; hence our assumption that all languages are enumerable is wrong.

Note:

- The total number of possible Turing Machines (encoded as a unique binary string) is less than size of Σ* for the binary alphabet. So, we can just enumerate all possible strings of Turing machines and discard invalid encodings.
  There are countably infinite numbers of Turing Machines in the world. But there are uncountably infinite number of formal languages. (we just proved)
- There exists a formal language for which we cannot construct a TM i.e. there exists a language which is not Recursively Enumerable.
- Set of Recursively Enumerable Languages is a proper subset of a set of all formal languages.

- Theorem: Let S be an infinite countable set. Its powerset 2^S is not countable.

Turing acceptable languages

1. Recursive language (Decidable Language)

A formal language is recursive if there exists a Turing machine that, **when given a finite sequence of symbols as input, always halts and accepts it if it belongs to the language and halts and rejects it otherwise**.

The class of all recursive languages is often called **REC.**

This type of language was not defined in the Chomsky hierarchy of (Chomsky 1959). All recursive languages are also recursively enumerable. All regular, context-free and context-sensitive languages are recursive.

Definitions

There are many equivalent major definitions for the concept of a recursive language:

1. A recursive formal language is a recursive subset in the set of all possible words over the alphabet of the language.
2. A recursive language is a formal language for which there exists a Turing machine that, when presented with any finite input string, halts and accepts if the string is in the language, and halts and rejects otherwise. The Turing machine always halts: it is known as a decider and is said to *decide* the recursive language.
3. **Recursive languages are also called Decidable Languages because a Turing Machine can decide membership in those languages (it can either accept or reject a string).**

By the second definition, any decision problem can be shown to be decidable by exhibiting an algorithm for it that terminates on all inputs. An undecidable problem is a problem that is not decidable.

Summary.

If L is a recursive language, then

- If $w \in L$ then a TM halts in a final state and accept the string.
- If $w \notin L$ then TM halts in a non-final state and reject the string.

Note:

- A language $L \subseteq \Sigma^*$ is called recursively enumerable if it is accepted by some Turing machine M.
- A language $L \subseteq \Sigma^*$ is called recursive if it is accepted by some Turing machine M which halts on all inputs.
- If a type of problem or, equivalently, a formal language has a decider Turing Machine, then it is said to be computable or decidable. In other words, all recursive languages are computable or decidable.
- If there was no halting problem, we could convert every acceptor machine to a decider, thereby making every recursively enumerable language recursive.

Recursive Languages are also Recursively Enumerable

Proof: If L is a recursive then there is TM which decides a member in language then

- M accepts x if x is in language L.
- M rejects on x if x is not in language L.

According to the definition, M can recognize the strings in language that are accepted on those strings.

Closure properties

Recursive languages are closed under the following operations. That is, if *L* and *P* are two recursive languages, then the following languages are recursive as well:

- the Kleene star L^* of *L*
- the concatenation L.P of *L* and *P*
- the union LUP
- the intersection L∩P.
- The complement of L
- The set difference L-P

The last property follows from the fact that the set difference can be expressed in terms of intersection and complement.

Examples of

As noted above, every **context-sensitive language** is recursive. Thus, a simple example of a recursive language is the set `L={abc, aabbcc, aaabbbccc, ...};` more formally, the set

$$L = \{a^n b^n c^n \mid n \geq 1\}$$

is context-sensitive and therefore recursive.

L= $\{a^n b^n c^n | n>=1\}$ is recursive because we can construct a Turing machine which will move to final state if the string is of the form $a^n b^n c^n$ else move to non-final state. So, the TM will always halt in this case.

A **context-sensitive grammar** for $L = \{a^n b^n c^n \mid n \geq 1\}$ is

$S \rightarrow aBSc \mid abc$
$Ba \rightarrow aB$
$Bb \rightarrow bb$

Try to derive aaabbbccc?

[S⇒ aBSc ⇒ aBaBScc ⇒ aBaBabccc ⇒ aaBBabccc ⇒ aaBaBbccc ⇒ aaBabbccc ⇒ aaaBbbccc ⇒ aaabbbccc]

Note:

- **Context-Sensitive Grammar**:
  - CSG Production rules constraints: αAβ→αγβ (where α, β, γ = string of terminals and/or non-terminals, A = Non-terminal)
- **A linear bounded automata (LBA) is a restricted form of Turing machine**.

  An LBA satisfies the following three conditions:
  - Its input alphabet includes two special symbols, serving as left and right endmarkers.
  - Its transitions may not print other symbols over the endmarkers.
  - Its transitions may neither move to the left of the left endmarker nor to the right of the right endmarker.

2. Recursively Enumerable language (Type-0 Language)

(also called partially decidable, semi decidable, Turing-acceptable or Turing-recognizable)

A language L is recursively enumerable if there exists a Turing Machine M such that L=L(M).

The class of all recursively enumerable languages is called **RE**.

Definitions

There are many equivalent definitions of a recursively enumerable language:

1. A recursively enumerable language is a recursively enumerable subset in the set of all possible words over the alphabet of the language, i.e., if there exists a Turing machine which will enumerate all valid strings of the language.

2. A recursively enumerable language is a formal language for which there exists a Turing machine (or other computable function) which will enumerate all valid strings of the language. Note that if the language is infinite, the enumerating algorithm provided can be chosen so that it avoids repetitions, since we can test whether the string produced for number *n* is "already" produced for a number which is less than *n*. If it already is produced, use the output for input *n*+1 instead (recursively), but again, test whether it is "new".

3. A recursively enumerable language is a formal language for which there exists a Turing machine (or other computable function) that will halt and accept when presented with any string in the language as input but may either halt and reject or loop forever when presented with a string not in the language. Contrast this to recursive languages, which require that the Turing machine halts in all cases.

4. **A language is called *Recursively Enumerable* if there is a Turing Machine that accepts on any input within the language**. **Recursively Enumerable Languages are also called *Recognizable* because a Turing Machine can recognize a string in the language (accept it). It might not be able to decide if a string is not in the language since the machine might loop for that input.**

Summary.

If L is a Recursively Enumerable language (or partially decidable), then

- **If w ∈ L then a TM halts in a final state and accept the string,**
- **If w ∉ L then TM**
  - **halts in a non-final state and reject the string or**
  - **never halt, i.e., it may go into an infinite loop.**

Closure properties

Recursively enumerable languages (REL) are closed under the following operations. That is, if L and P are two recursively enumerable languages, then the following languages are recursively enumerable as well:

- the Kleene star L^* of L
- the concatenation L.P of L and P
- the union LUP
- the intersection L∩P.

Recursively enumerable languages are not closed under set difference or complementation. The set difference L-P is recursively enumerable if P is recursive. If L is recursively enumerable, then the complement of is recursively enumerable if and only if L is also recursive.

Note:

- An undecidable language maybe a partially decidable language or something else but not decidable.
- Undecidable languages are not recursive languages, they may be subsets of Turing recognizable languages: i.e., such undecidable languages may be recursively enumerable.

- If a language is not even partially decidable, then there exists no Turing machine for that language.
- All regular, context-free, context-sensitive and recursive languages are recursively enumerable.

Examples of Recursively Enumerable Language

Some recursively enumerable languages that are not recursive (or undecidable) include:

1. **Language $HALT_{TM}$ = {< M, w > | M is a TM and M halts on input w }**

   (Recursively Enumerable/An Undecidable/Non-computable Problem)

   Answer: (in brief)

   - We know that, we can encode the Transition table of a Turing Machine as Description i.e., a unique binary string of finite length. Encode all Turing machines (i.e., every Turing machine is a binary string of finite length).
   - The total number of possible Turing Machines (i.e., Descriptions encoded in binary) is less than the size of Σ^* for the binary alphabet $\Sigma = \{0,1\}$.
   - There are countably infinite numbers of Turing Machines in the world.
   - Write down the encoded binary strings in the ascending order of string length and among strings of equal length, order them alphabetically. Hence every encoded string has some position in the sequence. The set of turing machines is therefore countable (or recursively enumerable).
   - **Given a description of a Turing machine and its input i.e., (<M>, w), there is no general algorithm (or U_{TM}) that decide correctly whether Turing machine will halt or run indefinitely forever, for all possible Turing Machine and input pairs.**
   - **Hence, the halting problem over Turing machines is undecidable (i.e., unsolvable also).**

2. **L_{PCP} = {< P > | P is an instance of the PCP with a solution/match}**

   (Recursively Enumerable/An Undecidable/Non-computable Problem)

   ### What is Post Correspondence Problem(PCP)?

   The Post correspondence problem is an undecidable decision problem that was introduced by Emil Post in 1946. Because it is simpler than the halting problem and the *Entscheidungsproblem* it is often used in proofs of undecidability.

   ### Definition of the problem

   Let Σ be an alphabet with at least two symbols. The input of the problem consists of 2-finite lists $x_1, x_2, \ldots, x_n$ and $y_1, y_2, \ldots, y_n$ of n-words over Σ.

   A solution to this problem is a sequence of indices, such that the concatenation of the words from the respective list in that sequence yields the same resultant string.

   The decision problem then is to decide whether such a solution exists or not, for the given two lists above. We will prove here that this problem is unsolvable by any algorithm.

   Note:

   - Every word in the list should appear at least once in the **resultant string**.
   - A word can appear any number of times in the **resultant string**.

Example 1: Consider the following two lists

	List-1	List-2
Word-1	a	baa
Word-2	ab	aa
Word-3	bba	bb

A solution to this problem would be the sequence (3, 2, 3, 1), because

	List1				
	Word-3	Word-2	Word-3	Word-1	
Result1 =	**bba**	**ab**	**bba**	**a**	**= bbaabbbaa**
Result2 =	**bb**	**aa**	**bb**	**baa**	**= bbaabbbaa**
	Word-3	Word-2	Word-3	Word-1	
	List2				

Furthermore, since (3, 2, 3, 1) is a solution, so are all of its "repetitions", such as (3, 2, 3, 1, 3, 2, 3, 1), etc., that is, when a solution exists, there are infinitely many solutions of this repetitive kind.

Example 2: Consider the following two lists

	List-1	List-2
Word-1	1	111
Word-2	10111	10
Word-3	10	0

Explanation:

- **Step-1:** **We will start with a word in which both list-1 and list-2 are starting with same number**, so we can start with either word-1 or word-2. Let's go with **word-2**, hence

  ```
  Result1 = 10111
  Result2 = 10
  ```

- **Step-2:** We need 1s in result2 to match 1s in result1 so we will go with **word-1**, hence

  ```
  Result1 = 10111.1   = 101111
  Result2 = 10.111    = 10111
  ```

- **Step-3:** There is extra 1 in result1 to match this 1 so we will go with **word-1**, hence

  ```
  Result1 = 101111.1 = 1011111
  Result2 = 10111.111= 10111111
  ```

- **Step-4:** Now there is extra 1 in result2 to match it we will add **word-3**, hence

  ```
  Result1 = 1011111.10 = 101111110
  Result2 = 10111111.0 = 101111110
  ```

- As you can see, strings are same. The final solution sequence is **2 1 1 3**

Example 3: Consider the following two lists

	List-1	List-2
Word-1	100	1
Word-2	0	100
Word-3	1	00

We can try unlimited combinations for the above problem but none of the combination will lead us to solution, thus this problem does not have solution.

Example 4: Consider the following two lists

	List-1	List-2
Word-1	abb	bba
Word-2	aa	aaa
Word-3	aaa	aa

Solution: 2 1 3

Example 5: Consider the following two lists

	List-1	List-2
Word-1	ab	a
Word-2	bab	ba
Word-3	bbaaa	bab

No solution

Example 6: Consider the following two lists

	List-1	List-2
Word-1	10	101
Word-2	011	11
Word-3	101	011

No solution

Example 7: Consider the following two lists

	List-1	List-2
Word-1	abab	ababaaa
Word-2	aaabbb	bb
Word-3	aab	baab
Word-4	ba	baa
Word-5	ab	ba
Word-6	aa	a

There is a solution for the string 123456:

Note: It turns out that only some instances of PCP have solutions and others have no solution (**we cannot solve them, we may enter into an infinite loop while solving**)

We can express the PCP as a language as:

$$L_{PCP} = \{< P > | P \text{ is an instance of the PCP with a solution/match}\}$$

For a given instance, the instance having a solution (also called a match) is the same as saying that the instance belongs to the language L_{PCP} above. Similarly, the instance not having a solution/match means it doesn't belong to L_{PCP}.

Undecidability of L_{PCP}:

As theorem says that L_{PCP} is undecidable. That is, there is no particular algorithm that determines whether any PCP has solution or not and hence, it is undecidable (i.e., unsolvable also).

Proof of L_{PCP} Undecidability: (in brief)

The most common proof for the undecidability of L_{PCP} describes an instance of PCP that can simulate the computation of an arbitrary Turing machine on a particular input. A match will occur if and only if the input would be accepted by the Turing machine. Because deciding if a Turing machine will accept an input is a basic undecidable problem, PCP cannot be decidable either.

3. Non-Recursively Enumerable language (Non computable)

Examples:

1. **For any non-empty Σ, there exist languages that are not recursively enumerable (proved above)**

   Note: There exists a recursively enumerable language whose complement is not recursively enumerable.

Note:

- If a language L and its complement L^c are both recursively enumerable, then both languages are recursive.
- If L is recursive, then L^c is also recursive, and consequently both are recursively enumerable.

References:

1. https://en.wikipedia.org/wiki/Recursively_enumerable_language
2. https://jlmartin.ku.edu/courses
3. https://www.geeksforgeeks.org/post-correspondence-problem/
4. Undecidable(PCP,UTM,Reduction).pdf from Prof. Preet Kanwal, PESU

Department of Computer Science and Engineering
PES University, Bangalore, India

UE23CS242A: Automata Formal Language and Logic

Prakash C O, Associate Professor, Department of CSE

Knowledge Representation using Logic

The aim of this section is to show

- how logic can be used to form representations of the world,
- how a process of inference can be used to derive new representations about the world and
- how these can be used by an intelligent agent to deduce what to do.

We require:

- A *formal language* to represent knowledge in a computer tractable form.
- *Reasoning* - Processes to manipulate this knowledge to deduce non-obvious facts.

Note:

- **A programming language is a formal language** comprising a set of strings that produce various kinds of machine code output.
- **Logic(noun):**
  - a sensible reason or way of thinking.
  - a particular way of thinking, especially one that is reasonable and based on good judgment.

Why logic (or logic programming)?

The challenge is to design a language which allows one to represent all the necessary knowledge. We need to be able to make statements about the world such as describing things - people, houses, theories etc; relations between things and properties of things.

Logic makes statements about the world which are true (or false) if the state of affairs it represents is the case (or not the case). Compared to natural languages (expressive but context sensitive) and programming languages (good for concrete data structures but not expressive) logic combines the advantages of natural languages and formal languages. Logic is:

- concise
- unambiguous
- context insensitive
- expressive
- effective for inferences

What is a Logic?

A logic is defined by the following:

1. **Syntax** - describes the possible configurations that constitute sentences.
2. **Semantics** - determines what facts in the world the sentences refer to i.e. the interpretation. Each sentence makes a claim about the world.

3. **Proof theory** - set of rules for generating new sentences that are necessarily true given that the old sentences are true. The relationship between sentences is called entailment. The semantics link these sentences (representation) to facts of the world. The proof can be used to determine new facts which follow from the old.

Logical representation

- Logical representation **involves using formal logic systems** like **propositional logic** and predicate logic **to represent knowledge in a structured, precise, and unambiguous way.**
- Logical representation allows AI systems to perform reasoning by applying rules of inference to derive conclusions from known facts.

Propositional Logic

- **Propositional logic is also known as sentential logic, statement logic, propositional calculus, and zeroth-order logic.**
- **Propositional logic is a branch of logic that studies ways of combining or altering statements or propositions to form more complicated statements or propositions.**
- **Propositional logic is a branch of mathematical logic that studies the logical relationships between propositions (or statements, sentences, assertions) taken as a whole, and connected via logical connectives.**

What is a Proposition?

- **A proposition is a declarative sentence/statement** (that is a sentence which declares a FACT) **which is either true or false but not both.**

Examples.

1) **"Drilling for oil caused dinosaurs to become extinct."** is a proposition.
2) **"Look out!"** is not a proposition.
3) **"How far is it to the next town?"** is not a proposition.
4) **"x + 2 = 2x"** is not a proposition.
5) **"x + 2 = 2x when x = 2"** is a proposition.
6) **3 + 2 = 5** is a simple mathematical proposition**.** Under the most common interpretation of the symbols in it, it is of course true.
7) **3 + 2 = 7** is also a proposition, even though it is false in the standard number system. Nothing says a proposition can't be false. Also, this equation could be true (and the previous one false) in a nonstandard number system.

Here are some further examples of propositions:

a) All cows are brown.
b) The Earth is further from the sun than Venus.
c) There is life on Mars.
d) $2 \times 2 = 5$.

Here are some sentences that are not propositions.

1. **"Do you want to go to the movies?"** Since a question is not a declarative sentence; hence, not a proposition.
2. **"Clean up your room."** Likewise, an imperative is not a declarative sentence; hence, not a proposition.
3. **"5x = 20 + x."** This is a declarative sentence, but unless x is assigned a value or is otherwise prescribed, the sentence neither true nor false, hence, not a proposition.
4. **"This sentence is false."** What happens if you assume this statement is true? false? This example is called a paradox and is not a proposition, because it is neither true nor false.
5. **Is anybody at home?** is not a proposition; questions are not declarative sentences.

Each proposition can be assigned one of two truth values. We use T or 1 for true and use F or 0 for false.

Atomic proposition:

- An atomic proposition is one whose truth or falsity does not depend on the truth or falsity of any other proposition. So, all the above propositions are atomic.

Propositional variables:

- Now, rather than write out propositions in full, we will represent them by using **propositional variables.**
- **It's a standard practice to use the lower-case roman letters p, q, r, … to stand for propositions**.

The Connectives (or Logical Operators)

Now, the study of atomic propositions is pretty boring. We therefore now introduce a number of connectives which will allow us to build up complex propositions.

Unary Operator negation:

a) **"not p",** $\neg p$.

Binary Operators:

a) **conjunction:** "p and q", $p \wedge q$.
b) **disjunction:** "p or q", $p \vee q$.
c) **implication:** "if p then q", $p \rightarrow q$.
d) **biconditional:** "p if and only if q", $p \leftrightarrow q$.

Conjunction (or AND) operation: (∧)

Any two propositions can be combined to form a third proposition called the conjunction of the original propositions.

Definition: If p and q are arbitrary propositions, then the conjunction of p and q is written **p ∧ q** and will be true iff both p and q are true.

We can summarise the operation of ∧ in a truth table. **The idea of a truth table for some formula is that it describes the behaviour of a formula under all possible interpretations of the primitive propositions that are included in the formula.** Let us write T for true, and F for false.

Then the truth table for p ∧ q is:

p	q	p ∧ q
F	F	F
F	T	F
T	F	F
T	T	T

Disjunction (or OR) operation: (∨)

Any two propositions can be combined to form a third proposition called the disjunction of the original propositions.

Definition: If p and q are arbitrary propositions, then the disjunction of p and q is written **p ∨ q** and will be true iff either p is true, or q is true, or both p and q are true.

The operation of ∨ is summarised in the following truth table:

p	q	p ∨ q
F	F	F
F	T	T
T	F	T
T	T	T

Conditional/Implication/if...then operation: (→)

Definition: If p and q are arbitrary propositions, then the conditional of p and q is written p → q and will be false precisely when p is true but q is false.

In essence, it is a statement that claims that **if one thing is true, then something else is true also**.

Here are a few examples of conditional statements:

- "If it is sunny, then we will go to the beach."
- "If the sky is clear, then we will be able to see the stars."
- "Studying for the test is a sufficient condition for passing the class."

Example 1: **p : This book is interesting.**
q : I am staying at home.
p → q : If this book is interesting, then I am staying at home.

Example 2: **p : The switch is on**
Q : The current flows
p → q : if the switch is on, then the current flows.

The operation of → is summarised in the following truth table:

p	q	p → q
F	F	T
F	T	T
T	F	F
T	T	T

p → q is logically equivalent to ¬pVq

p	¬p	q	¬pVq
F	T	F	T
F	T	T	T
T	F	F	F
T	F	T	T

Here's a typical list of ways we can express a logical-implication/Conditional operation on two arbitrary propositions p and q:

- **p implies q** **(if p is true then q is true)**
- **If p, then q**
- **If p, q**
- **q if p**
- **q when p**
- **q unless ¬p**
- **p is a sufficient condition for q**
- **q whenever p**
- **q follows p**
- **q is a necessary condition for p**

One way to think of the meaning of p → q is to consider it a contract that says **if the first condition is satisfied, then the second will also be satisfied.**

If the first condition, p, is not satisfied, then the condition of the contract is null and void. In this case, it does not matter if the second condition is satisfied or not, the contract is still upheld.

Note:

- In the statement "p implies q", q is a necessary condition for p. This is because the statement "p implies q" means that the logical meaning of p depends on the logical meaning of q. In other words, p can only be true if q is true, and p can only be false if q is false.
- In logic, the terms "necessity" and "sufficiency" are used to describe the relationship between two statements. In the statement "p implies q", p is a sufficient condition for q because p being true always implies that q is true.

Example 1:

p = you try hard for your exam.
q = you will succeed

p → q is "If you try hard for your exam, then you will succeed"

Case 1: **p = true** **q = true**
You tried hard for your exam and you succeeded
The Compound proposition **p → q is true**

Case 2: **p = true** **q = false**
You tried hard for your exam but you failed
The Compound proposition **p → q is false**

Case 3: **p = false** **q = true**
If you haven't tried hard for your exam, and you succeeded
The Compound proposition **p → q is true (How?)**
If p is true then only, we can argue about Compound proposition **p → q**

Example 2:

For example, suppose your friend tells you that if you meet her for lunch, she will give you a book she wants you to read. According to this statement, you would expect her to give you a book if you do go to meet her for lunch. But what if you do not meet her for lunch? She did not say anything about that possible situation, so she would not be breaking any kind of promise if she dropped the book off at your house that night or if she just decided not to give you the book at all. If either of these last two possibilities happens, we would still say the implication stated was true because she did not break her promise.

In the conditional statement p → q

- **p is called the Premise, Hypothesis,** or **the Antecedent.**
- **q is called the Conclusion** or **the Consequent.**

For the conditional statement p → q

- **q → p** is the **Converse.**
- **¬p → ¬q** is the **Inverse.**
- **¬q → ¬p** is the **Contrapositive.**

Example: If this book is interesting, then I am staying at home.

- **Converse : If I am staying at home, then this book is interesting.**
- **Inverse : If this book is not interesting, then I am not staying at home.**
- **Contrapositive : If I am not staying at home, then this book is not interesting.**

Exercise: **p** : "I will prove this by cases",
q : "There are more than 500 cases," and
r : "I can find another way."

(1) State (¬r ∨ ¬q) → p in simple English.
(2) State the converse of the statement in part 1 in simple English.
(3) State the inverse of the statement in part 1 in simple English.
(4) State the contrapositive of the statement in part 1 in simple English.

Note:

What does "Implies" mean in logic?

- Implies doesn't mean Suggestion or Causation.

For example:

- Red is a colour implies Bangalore is in Karnataka.
- 7 is even implies x is 3.

Both the statements are perfectly okay from the logic point of view. All Statements must have a truth value (even though it doesn't make sense).
https://www.youtube.com/watch?v=3RFamYOCEHA

Biconditional/iff (↔) operation

In logic, **the logical biconditional, also known as biimplication** or **bientailment**, is the logical connective used to conjoin two statements p and q to form the statement **"p if and only if q".**

Definition: If p and q are arbitrary propositions, then the biconditional of p and q is written: **p ↔ q**
and will be true iff either:

- p and q are both true; or
- p and q are both false.

The truth table for ↔ is:

p	q	p ↔ q
F	F	T
F	T	F
T	F	F
T	T	T

If p ↔ q is true, then p and q are said to be logically equivalent. They will be true under exactly the same circumstances.

p↔q is logically equivalent to both **(p→q)∧(q→p)** and **(p∧q)∨(¬p∧¬q)**, and the **XNOR** (exclusive nor) boolean operator, which means "both or neither".

p	q	p → q	q → p	(p→q)∧(q→p)
F	F	T	T	T
F	T	T	F	F
T	F	F	T	F
T	T	T	T	T

Exercise-01: Let p, q, and r be the propositions

- **p :** You have the flu.
- **q :** You miss the final examination.
- **r :** You pass the course.

Express the propositions as English sentences:

a) p → q
b) ¬q ↔ r
c) q → ¬r
d) (p → ¬r) ∨ (q → ¬r)
e) (p ∧ q) ∨ (¬q ∧ r)

Solution:

a) **p → q** : If you have the flu then you miss the final examination.
b) **¬q ↔ r** : You pass the course iff you do not miss the final examination.
c) **q → ¬r** : If you miss the final examination then you will not pass the course.
d) **(p → ¬r) ∨ (q → ¬r)** : If you have the flu then you will not pass the course or if you miss the final examination then you will not pass the course.
e) **(p ∧ q) ∨ (¬q ∧ r)** : You have the flu and you miss the final examination or you will not miss the final examination and you pass the course.

Exercise-02: Let p and q be the propositions

- **p:** You drive over 60 kmph.
- **q:** You get a speeding ticket.

Write the following propositions using p, q and logical connectives.

a) You do not drive over 60 kmph
b) You drive over 60 kmph, but you do not get a speeding ticket.
c) You will get a speeding ticket if you drive over 60 kmph.
d) If you do not drive over 60 kmph, then you will not get a speeding ticket.
e) You get a speeding ticket, but you do not drive over 60 kmph.

Solution:

a) You do not drive over 60 kmph : $\neg p$.

b) You drive over 60 kmph, but you do not get a speeding ticket : $p \wedge \neg q$.

c) You will get a speeding ticket if you drive over 60 kmph : $p \rightarrow q$

d) If you do not drive over 60 kmph, then you will not get a speeding ticket : $\neg p \rightarrow \neg q$

e) You get a speeding ticket, but you do not drive over 60 kmph : $q \wedge \neg p$.

Exercise-03: Determine the truth values of the following conditional and biconditional statements.

1) 2+2=4 if and only if 1+1=2.
2) 1+1=2 if and only if 2+3=4.
3) 0>1 if and only if 2>1.
4) If 1+1=2, then 2+2=5.
5) If 1+1=3, then 2+2=4.
6) If 1+1=3, then 2+2=5.
7) If 2+2=4, then 1+2=3.
8) 1+1=3 if and only if monkeys can fly.
9) If monkeys can fly, then 1+1=3
10) If 1+1=3, then unicorns exist.
11) If 1+1=3, then horse can fly.
12) If 1+1=2, then horse can fly.

Determine the truth values of the following conditional and biconditional statements.

1) 2+2=4 if and only if 1+1=2. True
2) 1+1=2 if and only if 2+3=4. False
3) 0>1 if and only if 2>1. False
4) If 1+1=2, then 2+2=5. False
5) If 1+1=3, then 2+2=4. True
6) If 1+1=3, then 2+2=5. True
7) If 2+2=4, then 1+2=3. True
8) 1+1=3 if and only if monkeys can fly. True
9) If monkeys can fly, then 1+1=3 True
10) If 1+1=3, then unicorns exist. True
11) If 1+1=3, then horse can fly. True
12) If 1+1=2, then horse can fly. False

Tautology:

- A tautology is a statement that is always true, regardless of the values of its variables.
- A tautology is a compound statement that is made up of two or more simple statements, joined by connectives and conditional words like "and", "or", "not", "if then", and "if and only if".
- A statement is a tautology if all of the column values in its truth table are true (T).

p	q	p→q	q→p	(p→q)∧(q→p)	p↔q	(p→q)∧(q→p) ↔ (p↔q)
F	F	T	T	T	T	T
F	T	T	F	F	F	T
T	F	F	T	F	F	T
T	T	T	T	T	T	T

Note:

- The following gives a hierarchy of precedence's for the operators of propositional logic.
  The ¬ operator has higher precedence than ∧; ∧ has higher precedence than ∨; and ∨ has higher precedence than →; and → has higher precedence that ↔.

Contradiction:

- Contradiction is a compound proposition that is always false no matter what the truth values of the propositions that occur in it.
- A statement that is false in all situations is called a contradiction. A contradiction is also known as a fallacy.

p	q	p → q	¬(p→q)	p∧¬q	¬(p∧¬q)	¬(p→q) ∧ ¬(p∧¬q)
F	F	T	F	F	T	F
F	T	T	F	F	T	F
T	F	F	T	T	F	F
T	T	T	F	F	T	F

- Here are some examples of tautologies and contradictions:
  - **(¬p∨¬q)∨(p∨¬q)**
  - **(p→q)∨(p∧¬q)**
  - **(p→q)∧(p∧¬q)**
  - **(¬p∧q)∧(¬q)**
  - **(¬p∧q)∨(¬q)**
  - **¬(p ∨ ¬p)**
  - **¬(p → p)**
  - **¬((p ∨ q) ↔ (q ∨ p))**

Logical equivalence

- The compound propositions A and B are called logically equivalent, if **A↔B** is a tautology.
- The notation A ≡ B denotes that A and B are logically equivalent (i.e., all of the final column values in their truth table are same).
- In other words, compound propositions that have the same truth values in all possible cases are called logically equivalent.

Note:

- The symbol ≡ is not a logical connective, hence "A ≡ B" is not a compound proposition.
- **"A ≡ B"** means **A↔B is a tautology.**

Logical equivalences involving conditional statements:

$p \rightarrow q \equiv \neg p \vee q$

$p \rightarrow q \equiv \neg q \rightarrow \neg p$

$p \vee q \equiv \neg p \rightarrow q$

$p \wedge q \equiv \neg(p \rightarrow \neg q)$

$\neg(p \rightarrow q) \equiv p \wedge \neg q$

$(p \rightarrow q) \wedge (p \rightarrow r) \equiv p \rightarrow (q \wedge r)$

$(p \rightarrow r) \wedge (q \rightarrow r) \equiv (p \vee q) \rightarrow r$

$(p \rightarrow q) \vee (p \rightarrow r) \equiv p \rightarrow (q \vee r)$

$(p \rightarrow r) \vee (q \rightarrow r) \equiv (p \wedge q) \rightarrow r$

Logical equivalences involving biconditional statements:

$p \leftrightarrow q \equiv (p \rightarrow q) \wedge (q \rightarrow p)$

$p \leftrightarrow q \equiv \neg p \leftrightarrow \neg q$

$p \leftrightarrow q \equiv (p \wedge q) \vee (\neg p \wedge \neg q)$

$\neg(p \leftrightarrow q) \equiv p \leftrightarrow \neg q$

Problem-1: Using truth table, show that **(p↔q) ≡ (p→q) ∧ (q→p).**

p	q	p↔q	(p→q) ∧ (q→p)	(p↔q) ↔ (p→q) ∧ (q→p)
F	F	T	T	T
F	T	F	F	T
T	F	F	F	T
T	T	T	T	T

Truth table demonstrates, (p↔q) ↔ (p→q) ∧ (q→p) is a tautology. Therefore, (p↔q) ≡ (p→q) ∧ (q→p)

Problem-2: Using truth table, show that **p→q ≡ ¬p∨q** (i.e., show that p→q and ¬p∨q are logically equivalent.)

p	q	¬p	p→q	¬p∨q	(p→q) ↔ (¬p∨q)
F	F	T	T	T	T
F	T	T	T	T	T
T	F	F	F	F	T
T	T	F	T	T	T

Truth table demonstrates, (p→q) ↔ (¬p∨q) is a tautology.

Therefore, (p→q) ≡ (¬p∨q)

Rules of Substitution (Laws of Logic)

Also called Rules of Equivalence, these are biconditional rules - i.e. they apply from L to R and also from R to L. They are tautologies. Hence, they can be applied to even parts of clauses (can be substituted).

Key Propositional Equivalences (or Properties of Logical Equivalence)

1. Identity Laws

- **p∧T ≡ p**
- **p∨F ≡ p**

2. Domination Laws

- **$p \vee T \equiv T$**
- **$p \wedge F \equiv F$**

3. Idempotent Laws

- **$p \vee p \equiv p$**
- **$p \wedge p \equiv p$**

4. Double Negation Law

- **$\neg(\neg p) \equiv p$**

5. Commutative Laws

- **$p \vee q \equiv q \vee p$**
- **$p \wedge q \equiv q \wedge p$**

6. Associative Laws

- **$(p \vee q) \vee r \equiv p \vee (q \vee r)$**
- **$(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$**

7. Distributive Laws

- **$p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$**
- **$p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$**

8. De Morgan's Laws

- **$\neg(p \wedge q) \equiv \neg p \vee \neg q$**
- **$\neg(p \vee q) \equiv \neg p \wedge \neg q$**

9. Absorption Laws

- **$p \vee (p \wedge q) \equiv p$**
- **$p \wedge (p \vee q) \equiv p$**

10. Negation Laws

- **$p \vee \neg p \equiv T$**
- **$p \wedge \neg p \equiv F$**

Exercise 1: Prove the following logical equivalences using the laws of logic (without using the truth table).

1) $\neg(p \rightarrow q) \equiv p \wedge \neg q$
2) $p \wedge (p \rightarrow q)) \equiv p \wedge q$
3) $\neg(p \vee (\neg p \wedge q)) \equiv \neg(p \vee q)$
4) $(p \rightarrow r) \vee (q \rightarrow r) \equiv (p \wedge q) \rightarrow r$

Solution: 3) $\neg(p \vee (\neg p \wedge q)) \equiv \neg(p \vee q)$

LHS = $\neg(p \vee (\neg p \wedge q))$ Apply Distributive Law
= $\neg((p \vee \neg p) \wedge (p \vee q))$
= $\neg((p \vee \neg p) \wedge (p \vee q))$ Apply Negation Law i.e., $(p \vee \neg p) \equiv T$
= $\neg(T \wedge (p \vee q))$ Apply Identity Law i.e., $p \wedge T \equiv p$
= $\neg(p \vee q)$
= RHS

Exercise 2: Prove the following expressions are tautologies using the laws of logic (without using the truth table)

1) $(p \wedge q) \rightarrow (p \vee q)$
2) $(p \wedge q) \rightarrow (p \rightarrow q)$
3) $(p \leftrightarrow q) \leftrightarrow (p \rightarrow q) \wedge (q \rightarrow p)$

Solution: 1) $(p \wedge q) \rightarrow (p \vee q)$

$(p \wedge q) \rightarrow (p \vee q) = \neg (p \wedge q) \vee (p \vee q)$

$= \neg p \vee \neg q \vee (p \vee q)$

$= \neg p \vee \neg q \vee p \vee q$ Apply Negation Law i.e., $(p \vee \neg p) \equiv T$

$= \neg p \vee p \vee \neg q \vee q$

$= T \vee T$

$= T$

Note:

- **entail(verb):** Involve (something) as a necessary or inevitable part or consequence.

Logical entailment ($\alpha \vDash \beta$):

- **Entailment in logic is a relationship between statements where the truth of one statement guarantees the truth of another.**
- It's a type of deductive reasoning, where the truth of the premises guarantees the truth of the conclusion.
- **In formal notation, entailment is represented as $\alpha \vDash \beta$, which means that whenever α is true, β must also be true.**
- The propositions (or statements) α and β are logically equivalent (i.e., $\alpha \equiv \beta$) if $\alpha \vDash \beta$ and $\beta \vDash \alpha$.
- **The rule of the form $p_1 \wedge p_2 \wedge \ldots \wedge p_n \rightarrow q$ is called a logical entailment, then it must be a Tautology.**
  Where p_1 , p_2 , … p_n are called the premises and q is called the conclusion. We, can say conclusion is entailed by the premises.
  We can also write the logical argument (or logical entailment) as

  p_1
  p_2
  .
  .
  p_n
  _____
  $\therefore q$

Examples:

- Sindhu won the game → Sindhu played the game
- Fido is a dog → Fido is a mammal
- If someone is poisoned, then they will get sick.
- If someone is currently living, then they can age.

Note 1:

- "Entails" refers to a stronger relationship in which one statement necessarily follows from another.
- **If a set of premises entails a conclusion, then it is impossible for the premises to be true while the conclusion is false.**
- Logical entailment provides another way to define validity: a valid argument is one in which the premises entail the conclusion.

- A statement q is said to be a propositional consequence of statements $p_1, p_2, \ldots, p_n$ iff the single statement $p_1 \wedge p_2 \wedge \ldots \wedge p_n \rightarrow q$ is a tautology.

Note 2: Inference

- Inference is something that uses facts to determine other facts. A conclusion reached on the basis of evidence and reasoning.
- **In logic, an inference is a process of deriving logical conclusions from premises known or assumed to be true.**
- In artificial intelligence, we need intelligent computers which can create new logic from old logic or by evidence, so generating the conclusions from evidence and facts is termed as Inference**.**

Note 3:

In propositional logic, **a premise is a proposition**, or **declarative statement**, that is used as evidence to support another proposition, or conclusion, in an argument:

Definition:

- **Premise:** A statement or idea that provides evidence for a conclusion.
- **Conclusion:** The logical result of the relationship between the premises

An argument is made up of a set of premises and a conclusion. The argument is only meaningful if all of its premises are true.

Inference Rules (aka Rules of Inference (aka Transformation rule)):

- **Inference Rules are logical tools used to derive conclusions from premises.**

  For example, the rule of inference called *modus ponens* takes two premises, one in the form "If p then q" and another in the form "p", and returns the conclusion "q".

  The rule is valid with respect to the semantics of classical logic, in the sense that if the premises are true (under an interpretation), then so is the conclusion.
- Rules of inference are logical principles that allow you to derive new propositions from existing ones in propositional logic.
- Rules of inference are used to validate arguments and are especially important in logical arguments and proofs.

Types of Inference rules

Inference Rules	Tautology	Name
p $p \rightarrow q$ $\therefore q$	$(p \wedge (p \rightarrow q)) \rightarrow q$ **Note:** A conclusion q is said to be logically entailed from statements p, p→q iff the single statement $p \wedge (p \rightarrow q) \rightarrow q$ is a tautology.	**Modus ponens**
$\neg q$ $p \rightarrow q$ $\therefore \neg p$	$(\neg q \wedge (p \rightarrow q)) \rightarrow \neg p$	**Modus tollens**
...	...	...
$p \vee q$ $\neg p \vee r$ $\therefore q \vee r$	$((p \vee q) \wedge (\neg p \vee r)) \rightarrow (q \vee r)$	**Resolution**

1) Modus Ponens and Modus Tollens

- **Modus Ponens can be summarized as "if *p* then *q*. *p* both are true. Therefore, *q* must also be true."**
- **Modus Tollens can be summarized as "if *p* then *q*. not *q*. Therefore, not *p* also."**

1) Consider the statement (or the argument): **If Marcus is a poet, then he is poor.**

Modus Ponens: **Marcus is a poet.**
If marcus is a poet, then he is poor.

∴ Marcus is poor.

From the above example, if we know that both premises "If Marcus is a poet, then he is poor" and "Marcus is a poet" are both true, then the conclusion "Marcus is poor" must also be true.

Modus Tollens: **Marcus is not poor.**
If Marcus is a poet, then he is poor.

∴ Marcus is not a poet.

p = Marcus is a poet. q = Marcus is poor.

p	q	p→q	p ∧ (p→q)	q	(p ∧ (p→q)) → q
F	F	T	F	F	T
F	T	T	F	T	T
T	F	F	F	F	T
T	T	T	T	T	T

Construction of a truth table will show that, Inference Rule Modus-Ponens is based on a tautology.

Note:

- The use of modus ponens in practice is as a rule of inference, rather than as a tautology. That is, **if we already have statements p and p→q, the modus ponens rule of inference says we can conclude q.**
- The fact that the statement (p ∧ (p→q)) → q is a tautology means that this inference rule is sound: if p and p→q are true, so is q.

2) Consider the statement: **If it is bright and sunny today, then I will wear my sunglasses.**

Modus Ponens: **It is bright and sunny today.**
If it is bright and sunny today, then I will wear my sunglasses.

∴ I will wear my sunglasses.

Modus Tollens: **I will not wear my sunglasses.**
If it is bright and sunny today, then I will wear my sunglasses.

∴ It is not bright and sunny today.

3) Consider the statement: **If an angle is inscribed in a semicircle, then it is a right angle.**

Modus Tollens: If an angle is inscribed in a semicircle, then it is a right angle; this angle is not a right angle; therefore, this angle is not inscribed in a semicircle.

Note:

- Modus ponens and modus tollens**, (**Latin: "method of affirming" and "method of denying") in propositional logic, two types of inference that can be drawn from a hypothetical proposition—*i.e.,* from a proposition of the form "If *p,* then *q*" (symbolically $p \rightarrow q$).
- An argument is valid if the conclusion is true whenever all the premises are true. An argument that is not valid is called a fallacy.

Exercises:

1. If this student is honest, she will not try to cheat when she takes a test.
   This student tried to cheat on a test.
   ∴ _____ by **modus** _____

2. If it is raining today, I will take my umbrella.
   It is raining today.
   ∴ _____ by **modus** _____

3. I always bring my lunch on Monday.
   I will buy my lunch today.
   ∴ _____ by **modus** _____

Answers:

1) ∴ **This student is not honest.** **by modus tollens**
2) ∴ **I will take my umbrella.** **by modus ponens**
3) ∴ **Today is not Monday** **by modus tollens**

2) Resolution Rule

- The **resolution rule** in propositional logic is **an inference rule that produces a new clause implied by two clauses containing complementary literals**. A literal is a propositional variable or the negation of a propositional variable. **The resulting clause (i.e., conclusion) contains all the literals that do not have complements.**

p ∨ q
¬p ∨ r } clauses
———————
∴ q ∨ r ← new clause (Conclusion).

Note: The dividing line stands for "entails".

The resolution inference rule takes two premises in the form of clauses ($p \vee q$) and ($\neg p \vee r$) and gives the clause ($q \vee r$) as a conclusion.

Exercise:

Q. No.	Clauses	Result after applying Resolution Rule
1	p ∨ q ∨ ¬r r ∨ s	Conclusion: **p ∨ q ∨ s**
2	p ∨ q ∨ r r ∨ ¬s ∨ ¬t	Not possible to apply Resolution rule, because no complimentary literals.
3	¬p ∨ ¬q ∨ r p ∨ q ∨ ¬r	No conclusion.

4	$p \rightarrow q$ p	Conclusion: **q**
5	$p \rightarrow q$ $\neg q$	Conclusion: **$\neg p$**

Note:

- **Literal:** A literal is either an atomic proposition or a negation of an atomic proposition. p, ¬p
- **Clause:** A clause is either a literal or a disjunction of literals. p, ¬p, p∨¬q
- $(p \wedge q) \vee (r \wedge s) = (p \vee r) \wedge (p \vee s) \wedge (q \vee r) \wedge (q \vee s)$ getting an AND of ORs by applying Distributive Law.
- **Conjunctive Normal Form (CNF): (an AND of ORs)**
  To convert a formula into a CNF.
  - Open up the implications (i.e., $p \rightarrow q$) to get ORs.
  - Get rid of negations from compound propositions.

- **Refutation**: The action of proving a statement to be wrong or false.

Note:

- We can use a truth table to show that an argument is valid. But, when we have more propositional variables involved in the argument, solving by truth table is too laborious (having just 30 variables will require over a billion rows in the truth table).
  That's why we need well-known argument forms to simplify a complex argument form at hand.
  These well-known valid argument forms are called Rules of Inference.

Resolution refutation: (A proof by contradiction using Resolution Inference rule.)

- **We use Resolution Refutation to prove that the conclusion logically entails from the premises.**

Resolution refutation: (Algorithm)

1. Convert all premises to CNF (an AND of ORs)
2. Negate the desired conclusion and then convert to CNF.
   It's a proof by Contradiction. (We assert that the thing that we are trying to prove is false, and then we try to derive a contradiction.)
3. List the disjunctive clauses got from step1 and step2 one below the other.
4. The resolution rule is applied to all possible pairs of clauses that contain complementary literals.
   After each application of the resolution rule, the resulting sentence is simplified by removing repeated literals and it will be considered for further resolution inferences.
   Apply resolution rule until either
   - Derive false i.e., empty clause (a contradiction)
     - Hence, it can be concluded that the initial conclusion (or inference) follows from the premises.
   - Can't apply any more to derive any more new clauses, then your desired conclusion can't be proved.

Note: In the resolution refutation process

• If we derive a contradiction, then the conclusion follows from the premises.
• If we can't apply any more, then the conclusion cannot be proved from the premises.

What if you can't apply the Resolution Rule anymore? Is there anything in particular that you can conclude?

- In fact, you can conclude that the thing that you were trying to prove can't be proved. So resolution refutation for propositional logic is a complete proof procedure. So if the thing that you're trying to prove is, in fact, entailed by the things that you've assumed, then you can prove it using resolution refutation.

- In the propositional case. It's more complicated in the first-order case, as we'll see. But **in the propositional case, it means that if you've applied the Resolution Rule and you can't apply it anymore, then your desired conclusion can't be proved.**
- It's guaranteed that you'll always either prove false, or run out of possible steps. **It's complete, because it always generates an answer.** Furthermore, **the process is sound: the answer is always correct.**

Example 1:

Use Resolution Refutation to prove that the conclusion logically entails from the premises.

Premises	Conclusion	Tautology
PVQ P→R Q→R	R	(PVQ)∧(P→R)∧(Q→R) → R

Answer:

No.	Clause	
1	PVQ	Given premise
2	¬PVR	Clause ¬PVR from premise P→R
3	¬QVR	Clause ¬QVR from premise Q→R
4	¬R	Negated Conclusion
5	QVR	Select two clauses that contain conflicting terms, i.e., from 1 and 2, Resolve P (i.e., cancel the terms that conflict) and combine left out literals by removing repeated literals.
6	R	From 3 and 5, Resolve Q and combine left out literals R and R as RVR. (RVR)≡R
7		From 4 and 6, Resolve R. **We got the empty clause, which is false.** **Hence, the conclusion logically entails from the premises**

And finally, resolving away R in lines 4 and 6, we get the empty clause, which is false. We'll often draw this little black box to indicate that we've reached the desired contradiction.

https://www.youtube.com/watch?v=ZYQkehjDLak

Note: (1) selecting two clauses that contain conflicting terms, (2) combining the terms contained in those two clauses, and then (3) canceling the terms that conflict. What makes resolution so important is that this one simple technique is capable of performing the same kinds of reasoning tasks as Modus Ponens, Modus Tollens, Chaining of Implication, and many other logical operations.

Example 2:

Use Resolution Refutation to prove that the conclusion logically entails from the premises.

Premises	Conclusion	Tautology
p	r	$(p \land (p \to q) \land ((p \to q) \to (q \to r))) \to r$
$p \to q$		
$(p \to q) \to (q \to r)$		

Solution:

1) Convert the premise **(p → q)** in to CNF

$(p \to q) = \neg p \lor q$

2) Convert the premise **(p → q) → (q → r)** in to CNF

$(p \rightarrow q) \rightarrow (q \rightarrow r)$ $= \neg(p \rightarrow q) \vee (q \rightarrow r)$

$= \neg(\neg p \vee q) \vee (\neg q \vee r)$

$= (\neg\neg p \wedge \neg q) \vee (\neg q \vee r)$

$= (p \wedge \neg q) \vee (\neg q \vee r)$

$= (p \vee \neg q \vee r) \wedge (\neg q \vee \neg q \vee r)$

$= (p \vee \neg q \vee r) \wedge (\neg q \vee r)$ **in CNF**

Resolution Refutation process:

No.	Clause	
1	p	Premise p
2	$\neg p \vee q$	Clause $(\neg p \vee q)$ from premise $p \rightarrow q$
3	$p \vee \neg q \vee r$	Clause $(p \vee \neg q \vee r)$ from premise $(p \rightarrow q) \rightarrow (q \rightarrow r)$
4	$\neg q \vee r$	Clause $(\neg q \vee r)$ from premise $(p \rightarrow q) \rightarrow (q \rightarrow r)$
5	$\neg r$	Negated Conclusion/Goal
6	q	Select two clauses that contain conflicting terms, i.e., from 1 and 2, Resolve p (i.e., cancel the terms that conflict) and combine left out literals by removing repeated literals.
7	r	From 4 and 6, Resolve q and combine left out literals.
8		From 5 and 7, Resolve r. **We got the empty clause, which is false.** (*false* $\wedge$ *anything* is false only, no need to resolve the left-out terms/literals) **Hence, the conclusion logically entails from the premises**

Exercise:

For the below problems, use Resolution Refutation to prove that the conclusion logically entails from the premises.

Q. No.	Premises	Conclusion	Tautology
1	$p \rightarrow q$ $r \rightarrow s$	$(p \vee r) \rightarrow (q \vee s)$	$((p \rightarrow q) \wedge (r \rightarrow s)) \rightarrow ((p \vee r) \rightarrow (q \vee s))$
2	p $\neg p$	r	$(p \wedge \neg p) \rightarrow r$
3	$a \vee \neg b$ $\neg c \vee b$ $\neg a$ $c \vee b \vee d$	d	$((a \vee \neg b) \wedge (\neg c \vee b) \wedge (\neg a) \wedge (c \vee b \vee d)) \rightarrow d$
4	$(p \rightarrow q) \rightarrow q$ $(p \rightarrow p) \rightarrow r$ $(r \rightarrow s) \rightarrow \neg(s \rightarrow q)$	r	$((p \rightarrow q) \rightarrow q) \wedge ((p \rightarrow p) \rightarrow r) \wedge ((r \rightarrow s) \rightarrow \neg(s \rightarrow q)) \rightarrow r$

The Wumpus World

[Wumpus World Simulator]

- **The wumpus world is a cave consisting of rooms** connected by passageways.
- **Lurking somewhere in the cave is the terrible wumpus**, a beast that eats anyone who enters its room.

- **The wumpus can be shot by an agent, but the agent has only one arrow.**
- **Some rooms contain bottomless pits that will trap anyone who wanders into these rooms** (except for the wumpus, which is too big to fall in).
- The only mitigating feature of this bleak environment is the possibility of finding a heap of gold.

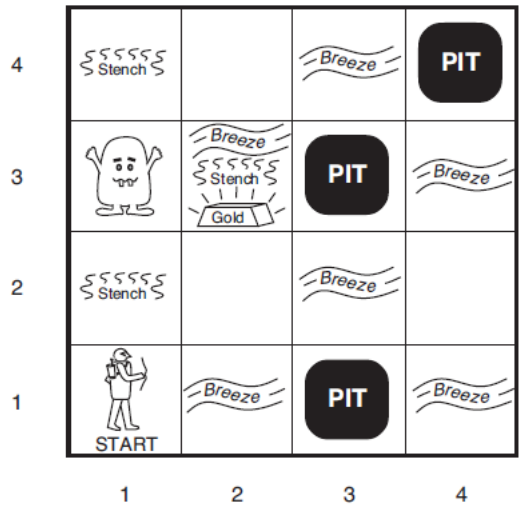


Figure 7.2 A typical wumpus world. The agent is in the bottom left corner, facing right.

The precise definition of the task environment is given, by the PEAS description:

- **Performance measure:**
  - +1000 for climbing out of the cave with the gold, –1000 for falling into a pit or being eaten by the wumpus, –1 for each action taken and –10 for using up the arrow.
  - The game ends either when the agent dies or when the agent climbs out of the cave.

- **Environment:**
  - A 4×4 grid of rooms.
  - **The agent always starts in the square labelled [1,1], facing to the right**.
  - The locations of the gold and the wumpus are chosen randomly, with a uniform distribution, from the squares other than the start square.
  - In addition, each square other than the start can be a pit, with probability 0.2.

- **Actuators:**
  - The agent can move *Forward*, *TurnLeft* by 90 degree, or *TurnRight* by 90 degree.
  - The agent dies a miserable death if it enters a square containing a pit or a live wumpus. (It is safe, albeit smelly, to enter a square with a dead wumpus.) If an agent tries to move forward and bumps into a wall, then the agent does not move.
  - The **action *Grab*** can be used to pick up the gold if it is in the same square as the agent.
  - The **action *Shoot*** can be used to fire an arrow in a straight line in the direction the agent is facing. The agent has only one arrow, so only the first *Shoot* action has any effect.
  - Finally, the **action *Climb*** can be used to climb out of the cave, but only from square [1,1].

- **Sensors:** **The agent has five sensors, each of which gives a single bit of information**:
  - In the square containing the wumpus and in the directly (not diagonally) adjacent squares, the agent will perceive a ***Stench*.**
  - In the squares directly adjacent to a pit, the agent will perceive a ***Breeze*.**
  - In the square where the gold is, the agent will perceive a ***Glitter*.**
  - When an agent walks into a wall, it will perceive a ***Bump*.**

- When the wumpus is killed, it emits a woeful ***Scream*** that can be perceived anywhere in the cave.

The percepts will be given to the agent program in the form of a list of five symbols; for example, if there is a stench and a breeze, but no glitter, bump, or scream, the agent program will get
[Stench, Breeze, None, None, None].

Let us watch a knowledge-based wumpus agent exploring the environment shown in Figure 7.2.

The agent's initial knowledge base contains the rules of the environment.

- **The first percept in cell [1,1] is [None, None, None, None, None], from which the agent can conclude that its neighboring squares, [1,2] and [2,1], are free of dangers**—they are OK.
- A cautious agent will move only into a square that it knows to be OK. Let us suppose the agent decides to move forward to [2,1]. The agent perceives a breeze in [2,1], so there must be a pit in a neighboring square. The pit cannot be in [1,1], by the rules of the game, so there must be a pit in [2,2] or [3,1] or both. So the prudent agent will turn around, go back to [1,1], and then proceed to [1,2].
- The agent perceives a stench in [1,2]. The stench in [1,2] means that there must be a wumpus nearby. But the wumpus cannot be in [1,1], by the rules of the game, and it cannot be in [2,2] (or the agent would have detected a stench when it was in [2,1]). Therefore, the agent can infer that the wumpus is in [1,3]. Moreover, the lack of a breeze in [1,2] implies that there is no pit in [2,2]. Yet the agent has already inferred that there must be a pit in either [2,2] or [3,1], so this means it must be in [3,1]. This is a fairly difficult inference, because it combines knowledge gained at different times in different places and relies on the lack of a percept to make one crucial step.
- The agent has now proved to itself that there is neither a pit nor a wumpus in [2,2], so it is OK to move there. We do not show the agent's state of knowledge at [2,2]; we just assume that the agent turns and moves to [2,3]. In [2,3], the agent detects a glitter, so it should grab the gold and then return home.
- **Note that in each case for which the agent draws a conclusion from the available information, that conclusion is *guaranteed* to be correct if the available information is correct. This is a fundamental property of logical reasoning.**

A simple knowledge base

Now that we have defined the semantics for propositional logic, **we can construct a knowledge base for the wumpus world**.
For now, we need the following symbols for each [x, y] location:

$\mathbf{P}_{x,y}$ is true if there is a pit in [x, y]. There are 16 propositions in total since it is a 4 X 4 grid.
$\mathbf{W}_{x,y}$ is true if there is a wumpus in [x, y], dead or alive. There are 16 propositions in total since it is a 4 X 4 grid.
$\mathbf{B}_{x,y}$ is true if the agent perceives a breeze in [x, y]. There are 16 propositions in total since it is a 4 X 4 grid.
$\mathbf{S}_{x,y}$ is true if the agent perceives a stench in [x, y]. There are 16 propositions in total since it is a 4 X 4 grid.

The sentences we write will suffice to derive $\neg P_{1,2}$ (**there is no pit in [1,2]**), We label each sentence R_i so that we can refer to them:

• **There is no pit in [1,1]:**

R1: $\neg P_{1,1}$

• **A square is breezy if and only if there is a pit in a neighboring square**. This has to be stated for each square; for now, we include just the relevant squares:

R2: $B_{1,1} \leftrightarrow (P_{1,2} \vee P_{2,1})$

R3: $B_{2,1} \leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$

• The preceding sentences are true in all wumpus worlds. **Now we include the breeze percepts for the first two squares visited in the specific world the agent is in**, leading up to the situation in Figure 7.3(b).

R4: $\neg B_{1,1}$
R5: $B_{2,1}$

A simple inference procedure

Assume that the Knowledge base (KB) is made up of the rules R1 to R5. The propositions involved in these rules are: B_{11}, B_{21}, P_{11}, P_{12}, P_{21}, P_{22}, P_{31}

Our goal now is to decide whether $KB \models \alpha$ for some sentence α. **For example, is $\neg P_{1,2}$ entailed by our KB?**
Our first algorithm for inference is a **model-checking approach** that is a direct implementation of the definition of entailment: enumerate the models, and check that α is true in every model in which KB is true.
Models are assignments of true or false to every proposition symbol.

Returning to our wumpus-world example, the relevant proposition symbols are $B_{1,1}$, $B_{2,1}$, $P_{1,1}$, $P_{1,2}$, $P_{2,1}$, $P_{2,2}$, and $P_{3,1}$. **With seven symbols, there are 2^7 =128 possible models; in three of these, KB is true** (Figure 7.9). In those three models, $\neg P_{1,2}$ is true, hence there is no pit in [1,2]. On the other hand, $P_{2,2}$ is true in two of the three models and false in one, so we cannot yet tell whether there is a pit in [2,2].

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	*false*	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*false*	*false*
false	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*false*	*true*	*false*	*false*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	*true*	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*false*	*true*	*true*	*false*
false	*true*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*false*	*true*	*false*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*true*	*false*	*false*	*true*	*false*	*false*	*true*	*true*	*false*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	*true*	*true*	*true*	*true*	*true*	*true*	*false*	*true*	*true*	*false*	*true*	*false*

Figure 7.9 A truth table constructed for the knowledge base given in the text. *KB* is true if R_1 through R_5 are true, which occurs in just 3 of the 128 rows (the ones underlined in the right-hand column). In all 3 rows, $P_{1,2}$ is false, so there is no pit in [1,2]. On the other hand, there might (or might not) be a pit in [2,2].

So far, we have shown **how to determine entailment** by ***model checking***: **enumerating models and showing that the sentence must hold in all models**. In this section, we show how entailment can be done by **theorem proving**—applying rules of inference directly to the sentences in our knowledge base to construct a proof of the desired sentence without consulting models.

If the number of models is large but the length of the proof is short, then theorem proving can be more efficient than model checking.

Propositional logic has several limitations, including:

- **Limited expressivity**
  Propositional logic can only represent simple statements with binary truth values (true or false).
  Propositional logic has very limited expressive power, unlike natural language.
  Example: We cannot express "pits cause breezes in adjacent squares" except by writing one sentence for each square.
- **Meaning in propositional logic is context-independent**
  unlike natural language, where meaning depends on context.

- **Can't handle quantifiers**
  Propositional logic can't handle quantifiers like "all", "some", or "none". For example, it can't represent the statement "all humans are mortal" concisely.
- **Can't establish truth**
  Propositional logic can't establish the truth of a proposition that isn't given as a premise.
- **Can't represent recursive structures**
  Propositional logic struggles to represent recursive structures, like lists or trees.
- **Can't represent temporal relationships**
  Propositional logic can't represent temporal relationships between events or states.
- **Can yield incorrect results**
  Propositional logic can sometimes yield incorrect results, like implying that an argument is invalid when it's actually valid.

Wumpus World and propositional logic

- **Find Pits in Wumpus world**
  - $B_{x,y} \leftrightarrow (P_{x,y+1} \vee P_{x,y-1} \vee P_{x+1,y} \vee P_{x-1,y})$ (Breeze next to Pit) 16 rules
- **Find Wumpus**
  - $S_{x,y} \leftrightarrow (W_{x,y+1} \vee W_{x,y-1} \vee W_{x+1,y} \vee W_{x-1,y})$ (stench next to Wumpus) 16 rules
- **At least one Wumpus in world**
  - $W_{1,1} \vee W_{1,2} \vee \ldots \vee W_{4,4}$ (at least 1 Wumpus) 1 rule
- **Keep track of location**
  - $L_{x,y} \wedge \text{FacingRight} \wedge \text{Forward} \rightarrow L_{x+1,y}$

First-Order Predicate Logic (FOPL)

- **First order logic** also known as **predicate logic**, **quantificational logic**, and **first-order predicate calculus.**
- **Propositional logic assumes that a world contains facts, whereas first-order logic (like natural language) assumes the world contains**
  - **Objects:** Objects can denote any real-world entity or any variable.
    Examples: A, B, colours, theories, circles, people, houses, numbers, theories, Ronald McDonald, baseball games, wars, centuries etc.
  - **Predicates (or Relations):** Relations/Predicates represent the links between different objects.
    Relations can be unary (relations defined for a single term) and n-ary(relations defined for n terms).
    Examples: blue, round (unary); friends, siblings (binary); etc.

    Predicates are functions that return true or false, representing properties of objects or relationships between them.
    Examples: Likes(Alice, Bob) indicates that Alice likes Bob.
    GreaterThan(x, 2) means that x is greater than 2.
  - **Functions:** Functions map their input object to the output object using their underlying relation.

Function is a special case of a relation, where given object is related to exactly one object. Function must provide output for all the inputs of the right type.
Input: Object or objects **Output:** Object
Examples:

Father(Punith)	denote the father of Punith.
Capital(Karnataka)	denote the capital of Karnataka
Sum(5, 25)	denote the sum of 5 and 25
Sqrt(5)	denote the square root of 5.

Indeed, almost any assertion (or statement) can be thought of as referring to objects and properties or relations. Some examples follow:

- **"One plus two equals three."**
  Objects: one, two, three, one plus two;
  Relation: equals;
  Function: plus.
  ("One plus two" is a name for the object that is obtained by applying the function "plus" to the objects "one" and "two." "Three" is another name for this object.)
- **"Squares neighboring the wumpus are smelly."**
  Objects: wumpus, squares;
  Property: smelly;
  Relation: neighboring.
- **"Evil King John ruled England in 1200."**
  Objects: John, England, 1200;
  Relation: ruled;
  Properties: evil, king.

Syntax of First Order Logic

- It represents the rules of representing any natural language construct in terms of First Order Logic.
- This involves the rules to describe any object and the relationships between different objects. These rules comprise rules for writing constants, variables, predicates, quantifiers, etc.

Basic Elements of First-Order Logic

The following table furnishes the basic elements of First Order Logic

Element	Example	Meaning
Constant	**1, 2, A, Raj, Mumbai, cat, ...**	Values that cannot be changed
Variables	**x, y, z, a, b, ...**	Can take up any value and can also change
Predicates (or Relations)	**Brother, Father, >, ...**	Defines a relationship between its input terms
Function	**sqrt, add, LeftLegOf, ...**	Computes a defined relation of input term
Connectives	∧, ∨, ¬, →, ↔	Used to form complex sentences using atomic sentences
Equality	=	Relational operator that checks equality
Quantifiers	∀, ∃	Imposes a quantity on the respective variable

Element	Example	Meaning
Term	**Constant Symbols or Function Symbols or Variables**	-

Atomic Sentences

Atomic sentences are the most basic expressions of First Order Logic. These sentences comprise a predicate followed by a set of terms inside a parenthesis.

Syntax: **Predicate (term-1, term-2, term-3,...)**

Examples:

Sisters(Geetha, Shwetha).
Polygon(Rectangle).

Complex Sentences

Complex sentences can be constructed by combining atomic sentences using connectives like AND (∧), OR (∨), NOT (¬), implies (→), if and only if (↔) etc.

Examples:

1) Sisters(Geetha, Shwetha) ∧ Sisters(Geetha, Rashmi).
2) ¬Sisters(Geetha, Divya)

Note:

- Any expression of First Order Logic can be broken into two main components, namely subject and predicate.
  - The subject is the main entity about which the expression describes.
  - Predicate depicts the relation between different subjects.
- For example, in the expression, Polygon(Rectangle), which is a translation of "Rectangle is a polygon." in First Order Logic, "Rectangle" is the subject, and "Polygon" represents a predicate.

Quantifiers in First-Order Logic

- **Quantifiers** in First Order Logic, as the name suggests, are **used to quantify any entity in a given environment.**
- **Quantification** refers to the identification of the total number of an entity that is present in the environment and satisfies a given expression in First Order Logic.
- Quantifiers enable us to determine the range and scope of a variable in a logical expression.

Two types of quantifiers are stated as follows.

1. **Universal Quantifier**
2. **Existential Quantifier**

Universal Quantifier

- Universal Quantifier is a symbol in a logical expression that signifies that the given expression is true in its range **for all instances of the concerned entity**.
- It is represented by the symbol ∀ (an inverted A).
- If x is a variable, then ∀*x* is read as **"For all x"** or **"For every x"** or **"For each x".**

Example: Let us take the sentence, **"All cats like fish".**

- Let us take a variable *x* which can take the value of "cat".

- Let us take a predicate cat(x) which is true if x is a cat.
- Similarly, let us take another predicate likes(x, y) which is true if x likes y.
- Therefore, using the universal quantifier ∀, we can write

  ∀x cat(x) → likes(x, fish).

  This expression is read as "For all x, if x is a cat, then x likes fish".

Existential Quantifier

- An **existential Quantifier** is a symbol in a logical expression that signifies that the given expression is true in its range **for at least one of the instances of the concerned entity.**
- It is represented by the symbol ∃ (an inverted E).
- If x is a variable, then ∃x is read as **"There exists x"** or **"For some x"** or **"For at least one x".**

Example: Let us take the sentence, **"Some students like ice cream"**.

- Let us take a variable x which can take the value of "student".
- Let us take a predicate student(x), which is true if x is a student.
- Similarly, let us take another predicate likes(x, y), which is true if x likes y.
- Therefore, using the existential quantifier ∃, we can write

  ∃x student(x) ∧ likes(x, ice-cream)

  This expression is read as, "There exists some x such that x is a student and also likes ice cream".

Properties of Quantifiers

Quantifiers in First Order Logic in AI follow some properties as stated below.

1. In the universal quantifier, **∀x ∀y** is equivalent to **∀y ∀x**.
2. In the existential quantifier, **∃x ∃y** is equivalent to **∃y ∃x**.
3. **∃x ∀y** is not equivalent to **∀y ∃x**.
   ∃x ∀y Loves(x, y) : "There is a person who loves everyone in the world"
   ∀y ∃x Loves(x, y) : "Everyone in the world is loved by at least one person"
4. **Quantifier Duality:**
   Each quantifier can be expressed using the other. **∀x Predicate(x)** is same as **¬∃x ¬Predicate(x).**

   ∀x Likes(x, IceCream) ≡ ¬∃x ¬Likes(x, IceCream)
   ∃x Likes(x, Broccoli) ≡ ¬∀x ¬Likes(x, Broccoli)

Points to Remember About Quantifiers in First-Order Logic

While using quantifiers in writing expressions for First Order Logic, we need to keep the following points in mind.

1. **The main connective for the universal quantifier ∀ is the implication (→).**
2. **The main connective for existential quantifier ∃ is AND (∧).**

Free and Bound Variables

There are two types of variables based upon their interaction with the quantifiers in a First Order Logic, namely free and bound variables.

1. **Free Variables:**
   Free variables are those **variables that do not come under the scope of the quantifier**.

For instance, in an expression ∀x ∃y **P(x, y, z)**, *z* is a free variable because it doesn't come under the scope of any quantifier.

2. **Bound Variables:**
   Bound variables are those **variables that occur inside the scope of the quantifier**. For instance, in an expression ∀x ∃y **P(x, y,** z**)**, x and y are bound variables because they occur inside the scope of the quantifiers.

Note: Variables stand for unspecified objects in the domain (where domain is a set of objects).

Example: Here x and y are variables

∀x,y sisters(x, y) → siblings(x, y)

Meaning: For every x and y, if x is a sister of y then x is a sibling of y.

∃x 5*5+x = 35

Meaning: There exists x such that result of 5*5+x is equal to 35.

Syntax of first-order logic:

$$
\begin{array}{rcl}
\textit{Sentence} & \rightarrow & \textit{AtomicSentence} \mid \textit{ComplexSentence} \\
\textit{AtomicSentence} & \rightarrow & \textit{Predicate} \mid \textit{Predicate}(\textit{Term}, \ldots) \mid \textit{Term} = \textit{Term} \\
\textit{ComplexSentence} & \rightarrow & (\ \textit{Sentence}\ ) \mid [\ \textit{Sentence}\ ] \\
& \mid & \neg\ \textit{Sentence} \\
& \mid & \textit{Sentence} \wedge \textit{Sentence} \\
& \mid & \textit{Sentence} \vee \textit{Sentence} \\
& \mid & \textit{Sentence} \Rightarrow \textit{Sentence} \\
& \mid & \textit{Sentence} \Leftrightarrow \textit{Sentence} \\
& \mid & \textit{Quantifier}\ \textit{Variable}, \ldots\ \textit{Sentence} \\
\textit{Term} & \rightarrow & \textit{Function}(\textit{Term}, \ldots) \\
& \mid & \textit{Constant} \\
& \mid & \textit{Variable} \\
\textit{Quantifier} & \rightarrow & \forall \mid \exists \\
\textit{Constant} & \rightarrow & A \mid X_1 \mid \textit{John} \mid \cdots \\
\textit{Variable} & \rightarrow & a \mid x \mid s \mid \cdots \\
\textit{Predicate} & \rightarrow & \textit{True} \mid \textit{False} \mid \textit{After} \mid \textit{Loves} \mid \textit{Raining} \mid \cdots \\
\textit{Function} & \rightarrow & \textit{Mother} \mid \textit{LeftLeg} \mid \cdots \\
\text{OPERATOR PRECEDENCE} & : & \neg, =, \wedge, \vee, \Rightarrow, \Leftrightarrow
\end{array}
$$

Figure 8.3 The syntax of first-order logic with equality, specified in Backus–Naur form (see page 1060 if you are not familiar with this notation). Operator precedences are specified, from highest to lowest. The precedence of quantifiers is such that a quantifier holds over everything to the right of it.

Exercise:

Let the domain be containing all the animals. Consider the predicates **Dog(x)**, **Mammal(x)** and **Cute(x)**. Write FOPL statements for 1)"All dogs are mammals" and 2)"Some dogs are cute".

Solution:

1)"All dogs are mammals": $\forall x\ (Dog(x) \rightarrow Mammal(x))$ (Note: $\forall x\ (Dog(x) \wedge Mammal(x))$ is wrong)

What does $\forall x\ (Dog(x) \rightarrow Mammal(x))$ mean?

$\forall x\ (Dog(x) \rightarrow Mammal(x)) \equiv \forall x\ (\neg Dog(x) \vee Mammal(x))$

Each animal is a non-dog or a mammal.

It means all dogs are mammals.

What does $\forall x\ (Dog(x) \wedge Mammal(x))$ mean?

$\forall x\ (Dog(x) \wedge Mammal(x)) \equiv (\forall x\ Dog(x)) \wedge (\forall x\ Mammal(x))$

All animals are dogs and all animals are mammals.

It is not same as "All dogs are mammals".

2) "Some dogs are cute": $\exists x\ (Dog(x) \wedge Cute(x))$ (Note: $\exists x\ (Dog(x) \rightarrow Cute(x))$ is wrong)

What does $\exists x\ (Dog(x) \rightarrow Cute(x))$ mean?

There exists an animal, if that animal is a Dog, then it is cute.

It means all dogs are cute.

It is not same as "Some dogs are Cute"

What does $\exists x\ (Dog(x) \wedge Cute(x))$ mean?

There exists some animal, which is a dog and is cute.

Please note the question says only few dogs are cute and not all.

Hence, there could exist a dog which is not cute.

Note:

Arity is a term used in logic, mathematics, and computer science to describe the number of arguments or operands a function, operation, or relation takes:

- In mathematics: Arity may also be called rank.
- In logic and philosophy: Arity may also be called adicity and degree.

Number of Possible Worlds/Models in FOPL:

- **In propositional logic**, suppose that you have 100 propositional variables (or symbols). How many possible worlds/models do you have?
  Answer: 2^{100} models (i.e., each row of the truth table represents one model)

- **In first-order logic**, how can we even count the number of possible worlds/models?
  The number of possible worlds is counted in terms of:
  - The number of objects
  - Mapping of Constant Symbols to Objects (Number of predicates and its arity)

  Answer: Number of possible models are unbounded. Checking entailment via Model Checking is not feasible as the number of combinations can become very large.

  Examples:

  1) Suppose we have: **Five constants, no functions** and **one predicate, that takes one argument**.

  How many possible worlds can we have?

  For each possible argument of the predicate, we must specify if the predicate returns true or false.

We have five possible arguments.
In total, we have 2^5=32 possible worlds/models (i.e., each row of the truth table is a model).

Model No.	P(V)	P(W)	P(X)	P(Y)	P(Z)
1	F	F	F	F	F
2	T	F	F	F	F
…	…	…	…	…	…
32	T	T	T	T	T

2) Suppose we have: **Five constants** (i.e., V, W, X, Y, Z), **no functions** and **one predicate that takes two arguments** (either of V, W, X, Y, Z).

How many possible worlds can we have?

For each possible combination of arguments of the predicate, we must specify if the predicate returns true or false.
We have 25 possible combinations of arguments.

The total number of possible worlds/models are 2^{25}

Model No.	P(V,V)	P(V,W)	P(V,X)	…	P(Z,Z)
1	F	F	F	…	F
2	T	F	F	…	F
…	…	…	…	…	…
2^{25}	T	T	T	…	T

Exercise:

1) Suppose we have: **Five constants** (i.e., V, W, X, Y, Z), **no functions**, **three predicates that takes two arguments** (either of V, W, X, Y, Z) and **One predicate that takes one argument.**

How many possible worlds can we have?

Solution:

For each possible combination of the arguments of each predicate, we must specify if the predicate returns true or false.
For predicate with two arguments, we have 25 combinations of arguments. We have three such predicates, so we must specify 75 values.
For predicate with one argument, we have 5 combinations of arguments. We have one such predicate, so we must specify 5 values.
In total, $2^{5+75} = 2^{80}$ possible worlds.

Applications of FOPL?

- In the wumpus world game of 4X4, to say that "pits cause breezes in adjacent squares" using propositional logic, we need 16 propositions.
- In first-order logic, how can we say the same thing?

  Objects: Pits, Squares, Wumpus, Gold, …

  Relations: Breeze, Adjacent, Stench, …

  $\forall x_1,y_1\ \text{Breeze}(x_1, y_1) \leftrightarrow (\exists x_2,y_2\ \text{Pit}(x_2, y_2) \wedge \text{Adjacent}(x_1, y_1, x_2, y_2))$

Convert the following Natural Language Sentences to FOPL statements.

1) Consider the following three sentences:

- "Each animal is an organism"
- "All animals are organisms"
- "If it is an animal then it is an organism"

This can be formalised as:

$\forall x$ (Animal(x) → Organism(x))

2) Mary loves everyone. [assuming domain D contains only humans]

$\forall x$ love (Mary, x)

3) Mary loves everyone. [assuming D contains both humans and non-humans, so we need to be explicit about 'everyone' as 'every *person*']

$\forall x$ (person(x) → love (Mary, x))

A wrong answer: $\forall x$ (person(x) & love (Mary, x)) This says that everything in the universe is a person and loves Mary.

4) No one talks. [assume D contains only humans unless specified otherwise.]

$\neg\exists x$ talk(x) or equivalently, $\forall x\neg$talk(x)

5) Everyone loves himself.

$\forall x$ love (x, x)

6) Everyone loves everyone.

$\forall x\forall y$ love (x, y)

7) Everyone loves everyone except himself. (= Everyone loves everyone else.)

$\forall x\forall y(\neg x = y \rightarrow$ love (x, y)) or **$\forall x\forall y(x \neq y \rightarrow$ love (x, y))**

Or maybe it should be this, which is not equivalent to the pair above:

$\forall x\forall y(\neg x = y \leftrightarrow$ love (x, y)) or **$\forall x\forall y(x \neq y \leftrightarrow$ love (x, y))**

The first pair allows an individual to also love himself; the second pair doesn't.

8) Every student smiles.

$\forall x$ (student(x) → smile(x))

9) Every student except George smiles.

$\forall x$ ((student(x) ∧ $x \neq$ George) → smile(x))

That formula allows the possibility that George smiles too; if we want to exclude it (this depends on what you believe about *except*; there are subtle differences and perhaps some indeterminacy among *except, besides, other than* and their nearest equivalents in other languages), then it should be the following, or something equivalent to it:

$\forall x$ ((student(x) → ($x \neq$ George ↔ smile(x)))

10) Everyone walks or talks.

$\forall x$ (walk (x) ∨ talk (x))

11) Every student walks or talks.

$\forall x\,(\text{student}(x) \rightarrow (\text{walk}(x) \vee \text{talk}(x)))$

12) Every student who walks talks.

$\forall x\,((\text{student}(x) \wedge \text{walk}(x)) \rightarrow \text{talk}(x))$ or $\forall x\,(\text{student}(x) \rightarrow (\text{walk}(x) \rightarrow \text{talk}(x)))$

13) Every student who loves Mary is happy.

$\forall x\,((\text{student}(x) \wedge \text{love}(x, \text{Mary})) \rightarrow \text{happy}(x))$

14) Every boy who loves Mary hates every boy who Mary loves.

$\forall x((\text{boy}(x) \wedge \text{love}(x, \text{Mary})) \rightarrow \forall y((\text{boy}(y) \wedge \text{love}(\text{Mary}, y)) \rightarrow \text{hate}(x,y)))$

15) Every boy who loves Mary hates every other boy who Mary loves.
(So if John loves Mary and Mary loves John, sentence 14 requires that John hates himself, but sentence 15 doesn't require that.)

$\forall x((\text{boy}(x) \wedge \text{love}(x, \text{Mary})) \rightarrow \forall y((\text{boy}(y) \wedge \text{love}(\text{Mary}, y) \wedge y \neq x) \rightarrow \text{hate}(x,y)))$

16) Using the following notations, express the given statements in FOPL

Q(x) : x is a rational number
R(x) : x is a real number
LESS(x, y) : x is less than y

Statements:
i) There exists a number that is rational.
ii) Every rational number is a real number
iii) For every number x, there exists a number y, which is greater than x.
iv) Some real numbers are rational numbers.

Solution:

(i) $(\exists x)\ Q(x)$

(ii) $(\forall x)\ (Q(x) \rightarrow R(x))$

(iii) $(\forall x)\ (\exists y)\ \text{LESS}(x, y)$

(iv) $(\exists x)\ (Q(x) \wedge R(x))$

17) Let C(x) mean "x is a used-car dealer," and H(x) mean "x is honest." Translate each of the following formulas into English:

(i) $(\exists x)\ C(x)$

(ii) $(\exists x)\ H(x)$

(iii) $\forall x\ (C(x) \rightarrow \neg H(x))$

(iv) $(\exists x)\ (C(x) \wedge H(x))$

Solution:

(i) There is (at least) one (person) who is a used-car dealer.

(ii) There is (at least) one (person) who is honest.

(iii) All used-car dealers are dishonest.

(iv) (At least) one used-car dealer is honest.

18) **Problem Description:**

- The domain of interest is the natural numbers, $\mathbb{N}$.
- There are objects 1, 2, 3, ...
- There are functions, addition and multiplication, as well as the square function, on this domain.
- There are predicates on this domain, "even," "odd," and "prime."
- There are relations between elements of this domain, "equal," "less than", and "divides."
- For our logical language, we will choose symbols 1, 2, 3, ... add, mul, div, square, even, odd, prime, and so on, to denote these things.

Consider some ordinary statements about the natural numbers, express the same in FOPL:

1. Every natural number is even or odd, but not both.
2. A natural number is even if and only if it is divisible by two.
3. If some natural number, x is even, then so is x^2.
4. A natural number x is even if and only if $x+1$ is odd.
5. Any prime number that is greater than 2 is odd.
6. For any three natural numbers x, y, and z, if x divides y and y divides z, then x divides z.
7. There exists an odd composite number.

Solution:

1. $\forall x((\text{even}(x) \vee \text{odd}(x)) \wedge \neg(\text{even}(x) \wedge \text{odd}(x)))$
2. $\forall x(\text{even}(x) \leftrightarrow \text{div}(x,2))$
3. $\forall x(\text{even}(x) \rightarrow \text{even}(x2))$
4. $\forall x(\text{even}(x) \leftrightarrow \text{odd}(x+1))$
5. $\forall x(\text{prime}(x) \wedge x>2 \rightarrow \text{odd}(x))$
6. $\forall x \forall y \forall z(\text{div}(x,y) \wedge \text{div}(y,z) \rightarrow \text{div}(x,z))$
7. $\exists x(\text{odd}(x) \wedge \text{composite}(x))$

Knowledge Representation using FOPL

Example 1: Let us look at an example called Box Solver. There are 5 boxes

Box-1 **Box-2** **Box-3** **Box-4** **Box-5**

Ann, Bill, Charlie, Don, Eric own one box each but we don't know which box.

Try to find each box's' owner if you know that:

- Ann and Bill have boxes with the same colour.
- Don and Eric have boxes with the same colour.
- Charlie and Don have boxes with the same size.
- Eric's box is smaller than Bill's.

Here is the Prolog code to solve the problem.

```
% First define numbers for each box.
box(1).
box(2). %Unary Predicate
box(3).
box(4).
box(5).
% Box number, color, size.
box_details(1,black,3).
box_details(2,black,1).
box_details(3,white,1). %Ternary Predicate
box_details(4,black,2).
box_details(5,white,3).
owners(A,B,C,D,E):- %Function implementation as a rule
box(A), box(B), box(C), box(D), box(E),
A\=B,A\=C,A\=D,A\=E,
B\=C,B\=D,B\=E, %Distinct values
C\=D,C\=E,
D\=E,
box_details(A,ColorA,_), box_details(B,ColorA,_), % Ann and Bill have same color
box_details(D,ColorD,_), box_details(E,ColorD,_), % Don and Eric have same color
box_details(C,_,SizeC), box_details(D,_,SizeC), % Charlie and Don have same size
box_details(E,_,SizeE), box_details(B,_,SizeB),
SizeE < SizeB. % Eric's box is smaller than Bill's
```

Here is the program in action;

?- owners(A,B,C,D,E)

A = 2,
B = 4,
C = 1,
D = 5,
E = 3

Example 2: Let us look at an example in **Kinship (or relationship) domain**

Here is the Prolog code to solve the problem.

```
/* facts */
male(amitabh).
male(abhishek).
male(agastya).
male(nikhil).
female(jaya).
female(aishwarya).
female(aaradhya).
female(shweta).
female(navya).
parent(amitabh, abhishek).
parent(jaya, abhishek).
parent(amitabh, shweta).
parent(abhishek, aaradhya).
parent(aishwarya, aaradhya).
parent(shweta, navya).
parent(shweta, agastya).
parent(nikhil, navya).
/* rules */
mother(M,X):- parent(M,X),female(M).
father(F,X):- parent(F,X), male(F).
grandparent(G, X):- parent(G,P), parent(P,X).
sister(S,X):- parent(Z,X), parent(Z,S), female(S), S\=X.
brother(B,X):- parent(P, B), parent(P, X),male(B), not(B=X).
aunt(X,Y) :- female(X), sister(X,Z), parent(Z,Y).
```

Here is the program in action;

?- grandparent(amitabh, aaradhya)

true

Knowledge Engineering in First-order logic

What is knowledge-engineering?

- **The process of constructing a knowledge-base in first-order logic is called as knowledge-engineering.**
- In knowledge-engineering, someone who investigates a particular domain, learns important concept of that domain, and generates a formal representation of the objects, is known as **knowledge engineer**.

This approach is mainly suitable for creating **special-purpose knowledge base**.

The knowledge-engineering process

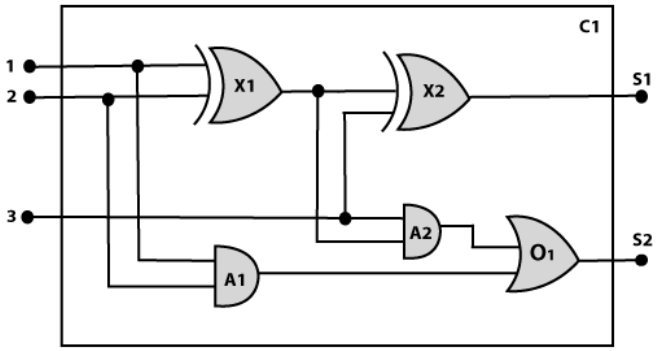
Following are some main steps of the knowledge-engineering process.

1) **Identify the task**
2) **Assemble the relevant knowledge**
3) **Decide on vocabulary**
4) **Encode general knowledge about the domain**
5) **Encode a description of the problem instance**

6) **Pose queries to the inference procedure and get answers**
7) **Debug the knowledge base**

The Knowledge engineering process in an Electronic Circuit Domain

Using the above steps, we will develop a knowledge base which will allow us to reason about digital circuit (**One-bit full adder**) which is given below



Possible queries:

- Does the circuit function properly?
- What gates are connected to the first input terminal?
- What would happen if one of the gates is broken?
- Etc..

1. Identify the task:

The first step of the process is to identify the task, and for the digital circuit, there are various reasoning tasks. At the first level or highest level, we will examine the functionality of the circuit:

- **Does the circuit add properly?**
- **What will be the output of gate A2, if all the inputs are high?**

At the second level, we will examine the circuit structure details such as:

- **Which gate is connected to the first input terminal?**
- **Does the circuit have feedback loops?**

2. Assemble the relevant knowledge:

In the second step, we will assemble the relevant knowledge which is required for digital circuits. So for digital circuits, we have the following required knowledge:

- Logic circuits are made up of wires and gates.
- Signal flows through wires to the input terminal of the gate, and each gate produces the corresponding output which flows further.
- In this logic circuit, there are four types of gates used: **AND, OR, XOR, and NOT**.
- All these gates have one output terminal and two input terminals (except NOT gate, it has one input terminal).

3. Decide on vocabulary:

The next step of the process is to select **functions, predicate, and constants** to represent the circuits, terminals, signals, and gates.

- Firstly, we will distinguish the gates from each other and from other objects.
- Each gate is represented as an object which is named by a constant, such as, **Gate(X1)** (returns true if X1 is a gate).
- The functionality of each gate is determined by its type, which is taken as constants such as **AND, OR, XOR, or NOT**.
- Circuits will be identified by a predicate: **Circuit (C1)**. (returns true if C1 is a circuit)
- For the terminal, we will use predicate: **Terminal(X)**. (returns true if X is a Terminal)

- For gate input, we will use the function **In(1, X1)** for denoting the first input terminal of the gate, (In(n, X1): returns nth Input Terminal of a Gate X1) and for output terminal we will use **Out(1, X1)**.
- The function **Arity(c, i, j)** is used to denote that circuit c has i input, j output. (Arity(c, i, j): returns true if c (gate or circuit) has i input terminals and j output terminals)
- The connectivity between gates can be represented by predicate **Connected(Out(1, X1), In(1, X1))**.
- We use a unary predicate **On(t)**, which is true if the signal at a terminal is on.

4. Encode general knowledge about the domain:

To encode the general knowledge about the logic circuit, we need some following rules:

- If two terminals are connected then they have the same input signal, it can be represented as:

  $\forall t1, t2\ \text{Terminal}(t1) \wedge \text{Terminal}(t2) \wedge \text{Connected}(t1, t2) \rightarrow \text{Signal}(t1) = \text{Signal}(2).$
- Signal at every terminal will have either value 0 or 1, it will be represented as:

  $\forall t\ \text{Terminal}(t) \rightarrow \text{Signal}(t) = 1 \vee \text{Signal}(t) = 0.$
- Connected predicates are commutative:

  $\forall t1, t2\ \text{Connected}(t1, t2) \rightarrow \text{Connected}(t2, t1).$
- Representation of types of gates:

  $\forall g\ \text{Gate}(g) \wedge r = \text{Type}(g) \rightarrow r = \text{OR} \vee r = \text{AND} \vee r = \text{XOR} \vee r = \text{NOT}.$
- Output of AND gate will be zero if and only if any of its input is zero.

  $\forall g\ \text{Gate}(g) \wedge \text{Type}(g) = \text{AND} \rightarrow \text{Signal}(\text{Out}(1, g)) = 0 \leftrightarrow \exists n\ \text{Signal}(\text{In}(n, g)) = 0.$
- Output of OR gate is 1 if and only if any of its input is 1:

  $\forall g\ \text{Gate}(g) \wedge \text{Type}(g) = \text{OR} \rightarrow \text{Signal}(\text{Out}(1, g)) = 1 \leftrightarrow \exists n\ \text{Signal}(\text{In}(n, g)) = 1$
- Output of XOR gate is 1 if and only if its inputs are different:

  $\forall g\ \text{Gate}(g) \wedge \text{Type}(g) = \text{XOR} \rightarrow \text{Signal}(\text{Out}(1, g)) = 1 \leftrightarrow \text{Signal}(\text{In}(1, g)) \neq \text{Signal}(\text{In}(2, g)).$
- Output of NOT gate is invert of its input:

  $\forall g\ \text{Gate}(g) \wedge \text{Type}(g) = \text{NOT} \rightarrow \text{Signal}(\text{In}(1, g)) \neq \text{Signal}(\text{Out}(1, g)).$
- All the gates in the above circuit have two inputs and one output (except NOT gate).

  $\forall g\ \text{Gate}(g) \wedge \text{Type}(g) = \text{NOT} \rightarrow \text{Arity}(g, 1, 1)$

  $\forall g\ \text{Gate}(g) \wedge r = \text{Type}(g) \wedge (r = \text{AND} \vee r = \text{OR} \vee r = \text{XOR}) \rightarrow \text{Arity}(g, 2, 1).$
- All gates are logic circuits:

  $\forall g\ \text{Gate}(g) \rightarrow \text{Circuit}(g).$

5. Encode a description of the problem instance:

Now we encode problem of circuit C1, firstly we categorize the circuit and its gate components. This step is easy if ontology about the problem is already thought. This step involves the writing simple atomics sentences of instances of concepts, which is known as ontology.

For the given circuit C1, we can encode the problem instance in atomic sentences as below:

Since in the circuit there are two XOR, two AND, and one OR gate so atomic sentences for these gates will be:

1. **For XOR gate: Type(X1)= XOR, Type(X2) = XOR**
2. **For AND gate: Type(A1) = AND, Type(A2)= AND**
3. **For OR gate: Type (O1) = OR.**

And then represent the connections between all the gates.

6. Pose queries to the inference procedure and get answers:

In this step, we will find all the possible set of values of all the terminal for the adder circuit. The first query will be:

What should be the combination of input which would generate the first output of circuit C1, as 0 and a second output to be 1?

∃i1, i2, i3 Signal(In(1, C1))=i1 ∧ Signal(In(2, C1))=i2 ∧ Signal(In(3, C1))= i3 ∧ Signal (Out(1, C1)) =0 ∧ Signal (Out(2, C1))=1

7. Debug the knowledge base:

Now we will debug the knowledge base, and this is the last step of the complete process. In this step, we will try to debug the issues of knowledge base.

In the knowledge base, we may have omitted assertions like 1 ≠ 0.

References:

1. LECTURE 7: PROPOSITIONAL LOGIC (1) by Mike Wooldridge.
2. https://plato.stanford.edu/entries/logic-propositional/
3. https://iep.utm.edu/propositional-logic-sentential-logic/#:¬:text=Propositional%20logic%2C%20also%20known%20as%20sentential%20logic%2C%20is%20that%20branch,more%20complicated%20statements%20or%20propositions.
4. https://ocw.mit.edu/courses/6-825-techniques-in-artificial-intelligence-sma-5504-fall-2002/3439d481b8e3d5abecc1403caf1ca89d_Lecture7FinalPart1.pdf
5. http://watson.latech.edu/book/intelligence/intelligenceApproaches2b2.html
6. https://eng.libretexts.org/Bookshelves/Computer_Science/Programming_and_Computation_Fundamentals/An_Introduction_to_Ontology_Engineering_(Keet)/03%3A_First_Order_Logic_and_Automated_Reasoning_in_a_Nutshell/3.01%3A_First_Order_Logic_Syntax_and_Semantics

---

Department of Computer Science and Engineering
PES University, Bangalore, India

UE23CS242A: Automata Formal Language and Logic

Prakash C O, Associate Professor, Department of CSE

Programming Paradigms

- **Paradigm** can also be termed as **a method to solve some problem or do some tasks**.
- **Programming paradigm** is **an approach to solve problem using some programming language** or also we can say it is a method to solve a problem using tools and techniques that are available to us following some approach.
- There are lots of programming languages that are known but all of them need to follow some strategy when they are implemented and this methodology/strategy is called as paradigms.

Apart from varieties of programming language there are lots of paradigms to fulfil each and every demand.

- Programming paradigms are of different ways or styles in which a given program or programming language can be organized.
- Each paradigm consists of certain structures, features, and opinions about how common programming problems should be tackled.

The question of why are there many different programming paradigms is similar to why are there many programming languages.

- Certain paradigms are better suited for certain types of problems, so it makes sense to use different paradigms for different kinds of projects.

Programming Paradigms

1. **Imperative programming paradigm**
    a) Procedural programming paradigm
    b) Object oriented programming
    c) Parallel processing approach
2. **Declarative programming paradigm**
    a) Logic programming paradigm
    b) Functional programming paradigms
    c) Database/Data driven programming approach

2. Declarative programming paradigm:

- It is divided as Logic, Functional, Database.

- The declarative programming is **a style of building programs that expresses logic of computation without talking about its control flow.**
- **The focus is on what needs to be done rather how it should be done basically emphasize on what code is actually doing.** It just declares the result we want rather how it has been produced.

  This is the only difference between **imperative (how to do)** and **declarative (what to do)** programming paradigms.

a) Logic programming paradigm

- The logic programming paradigm is **a programming approach that uses formal logic to represent and reason about knowledge**.

  Logic programs are written as a set of rules and facts that define relationships between entities, and **a query language to ask questions and get answers.**
- Logic programming is a declarative programming paradigm, which means it focuses on describing the problem, rather than specifying the steps to solve it.
- Logic programs **use logical reasoning to solve problems by applying logical relationships between facts and queries.**
- Logic programming defines an **inference mechanism** that derives new facts and rules from existing ones.
- **A knowledge base (KB)** in logic programming is a collection of facts and rules about a problem domain:
  Facts: Describe properties of entities (facts represent what is known about the world)
  Rules: Describe how new knowledge can be concluded from existing facts
  A KB can be used to store complex structured data and **answer questions about what is known**.
- In logical programming the main emphasize is on knowledge base and the problem. The execution of the program is very much like proof of mathematical statement, e.g., Prolog

Note:

- Formal logic is a fundamental part of computer science that **provides a framework for designing and understanding complex systems.**
- **Formal Logic is a set of rules for making deductions** based on symbolically representing objects and relationships, allowing for logical reasoning and making inferences about situations by determining what statements must be true.
- **Formal logic is the study of deductively valid inferences or logical truths.** It examines how conclusions follow from premises based on the structure of arguments alone, independent of their topic and content.
- **Formal logic:** A precise language used to represent knowledge in AI.

Prolog: Programming in Logic

- It is a logic programming language
- It's a declarative programming language(expresses the logic of a computation without describing its control flow)
- It is based on the principles of First Order Predicate Logic (FOPL)
- It is well-suited for developing logic-based AI applications.

In Prolog, the program logic is expressed in terms of relations, represented as facts and rules. A computation is initiated by running a query over these relations.

In prolog, we declare some facts. These facts constitute the Knowledge Base of the system. We can query against the Knowledge Base. We get output as affirmative if our query is already in the knowledge Base or it is implied by Knowledge Base, otherwise we get output as negative. So, Knowledge Base can be considered similar to database, against which we can query.

Note:

- Fact(noun) - something that you know has happened or is true (true things; reality). Something that is known to have happened or to exist, especially something for which proof exists, or about which there is information

Prolog Facts

Prolog facts are expressed in definite pattern.

- Facts contain entities and their relation.
- Entities are written within the parenthesis separated by comma (,).
- Their relation is expressed at the start and outside the parenthesis.
- Every fact/rule ends with a dot(.).

So, a Prolog fact format goes as follows:

relation(entity$_1$, entity$_2$, ...entity$_n$).

- Facts are statements that are assumed to be true.
- Facts are written using a predicate name followed by a list of arguments/entities enclosed in parentheses.
- A fact is a predicate expression that makes a declarative statement about the problem domain.

Examples:

Facts	These facts can be interpreted as
`friends(raj, praveen).`	`raj and praveen are friends.`
`singer(Shreya).`	`Shreya is a singer.`
`odd_number(7).`	`7 is an odd number.`
`capital_of(paris, france).`	`states that paris is the capital of France`
`woman(Aishwarya).`	`Aishwarya is a woman.`
`playsAirGuitar(Pratham).`	`Pratham plays AirGuitar`
`rockConcert.`	`The proposition rockConcert have been written so that the first letter is in lower-case.`

Once you have defined your facts in a Prolog program, you can use them to automatically infer solutions to problems.

For example, you could ask the interpreter to find the capital of france by using the following query:

capital_of(X, france)?

In this query, the interpreter will use the capital_of/2 fact that you defined earlier to determine that the capital of France is Paris, and it will return the value paris as the solution.

Prolog Rules

- Rules are logical statements that describe the relationships between different facts.
- A rule is a predicate expression that uses logical implication (:-) to describe a relationship among facts.
- Rules contain conditional clauses. To satisfy a rule all conditions should be met.

Rules are used to specify the conditions that must be met in order for a certain fact to be true.

In Prolog, **rules are written using the predicate name followed by a list of arguments enclosed in parentheses, followed by a colon and a hyphen (:-) and the body of the rule.**

For example: not(X,Y) :- man(X), woman(Y).

So, a rule format goes as follows:

Rule format	Rule interpretation
`A :- B1,B2,B3,.....Bn.`	`if B1,B2,…,Bn then A  or  B1 ∧ B2 ∧ … ∧ Bn → A`
`A :- B1;B2;B3;.....Bn.`	`B1 ∨ B2 ∨ B3 ∨ … ∨ Bn → A`

Example:

Rule	The rule can be interpreted as:
grandfather(X, Y) :- father(X, Z), parent(Z, Y)	This rule says that, for X to be the grandfather of Y, Z should be a parent of Y and X should be father of Z
eats(Person, Thing) :- likes(Person, Thing), food(Thing)	This says that a Person eats a Thing if the Person likes the Thing, and the Thing is food.
friends(X, Y) :- likes(X,Y), likes(Y,X).	X and Y are friends if they like each other
hates(X, Y) :- not(likes(X,Y)).	X hates Y if X does not like Y.
enemies(X, Y) :- not(likes(X,Y)), not(likes(Y,X)).	X and Y are enemies if they don't like each other

A rule in Prolog is a clause, normally with one or more variables in it. Normally, rules have a head, neck and body.

Sometimes, in a "rule", the body may be empty:

member(Item, [Item|Rest])

This says that the Item is a member of any list (the second argument) of which it is the *first* member. Here, no special condition needs to be satisfied to make the head true - it is always true.

Prolog Query

Queries are used to ask the interpreter to find solutions to problems based on the rules and facts in the program.

In Prolog, queries are written using the same syntax as facts preceded by symbols (?-).

So, a query format goes as follows:

```
?- B1, . . . , Bn
```

Prolog Variables

Variables are used to represent values that can change or be determined by the interpreter. In Prolog, variables are written using a name that begins with an uppercase letter, such as X or Y.

Variables can be used in both facts and rules to represent values that are not known at the time the program is written.

For example, the following fact uses a variable to represent the capital of a country:

Capital_of(Country, Capital)

In this case, the fact states that the **Capital** of a given Country is unknown, and the interpreter will use other facts and rules to determine the value of **Capital** when a query is made.

Note:

- The Facts and rules form the knowledge base
- The following terms are used interchangeably:
  fact, sentence, axiom - data which is given without being derived from other facts/sentences.
- Deriving new sentences from existing facts is called **inferencing.**
- There are only three basic constructs in Prolog: facts, rules, and queries.
- A collection of facts and rules is called a knowledge base (or a database) and **Prolog programming is all about writing knowledge bases**. That is, Prolog programs simply are knowledge bases, collections of facts and rules which describe some collection of relationships that we find interesting.

Program-1: (or Knowledgebase-1)

happy(Ram).
listens2Music(Krishna).
listens2Music(Ram) :- happy(Ram).
playsAirGuitar(Krishna) :- listens2Music(Krishna).
playsAirGuitar(Ram) :- listens2Music(Ram).

There are two facts in the above program, **listens2Music**(Krishna) and **happy**(Ram) . The last three items it contains are rules.

- Rules state information that is conditionally true of the situation of interest. For example, the first rule says that Ram listens to music if he is happy, and the last rule says that Ram plays air guitar if he listens to music.
- More generally, the **:-** should be read as "**if**", or "**is implied by**".
- The part on the left hand side of the :- is called the **head of the rule**, the part on the right hand side is called **the body of the rule**.
- So, in general rules say: **if the body of the rule is true, then the head of the rule is true too**.
- And now for the key point: If a knowledge base contains a rule (i.e., ***head :- body)***, and Prolog knows that body follows from the information in the knowledge base, then Prolog can infer head.

1) Let's consider an example. Suppose we ask whether Krishna plays air guitar:

   ?- playsAirGuitar(Krishna).

   Prolog will respond yes. Why? Well, although **it can't find playsAirGuitar(Krishna)** as a fact explicitly recorded in KB1, it can find the rule

   playsAirGuitar(Krishna):- listens2Music(Krishna).

   Moreover, KB1 also contains the fact **listens2Music(Krishna)** . Hence Prolog can use the rule of modus ponens to deduce that **playsAirGuitar(Krishna)** .

2) Our next example shows that Prolog can chain together uses of modus ponens. Suppose we ask:

?- playsAirGuitar(Ram).

Prolog would respond yes. Why? Well, first of all, by using the fact **happy(Ram)** and the rule

listens2Music(Ram):- happy(Ram).

Prolog can deduce the new fact **listens2Music(Ram)** .
This new fact is not explicitly recorded in the knowledge base — it is only implicitly present (it is inferred knowledge). Nonetheless, Prolog can then use it just like an explicitly recorded fact. In particular, from this inferred fact and the rule.

playsAirGuitar(Ram):- listens2Music(Ram).

it can deduce playsAirGuitar(Ram) , which is what we asked it.
Summing up: any fact produced by an application of modus ponens can be used as input to further rules. By chaining together applications of modus ponens in this way, Prolog is able to retrieve information that logically follows from the rules and facts recorded in the knowledge base.

- The facts and rules contained in a knowledge base are called clauses. Thus KB1 contains five clauses, namely three rules and two facts. Another way of looking at KB1 is to say that it consists of three predicates (or procedures). The three predicates are:
  - listens2Music
  - happy
  - playsAirGuitar
- The happy predicate is defined using a single clause (a fact).
  The listens2Music and playsAirGuitar predicates are each defined using two clauses (in one case, two rules, and in the other case, one rule and one fact). It is a good idea to think about Prolog programs in terms of the predicates they contain. In essence, the predicates are the concepts we find important, and the various clauses we write down concerning them are our attempts to pin down what they mean and how they are inter-related.
- One final remark. **We can view a fact as a rule with an empty body**. That is, we can think of facts as conditionals that do not have any antecedent conditions, or degenerate rules.

Program 2:

For instance, **Socrates is a man, all men are mortal,** and **therefore Socrates is mortal**.

The following is a simple Prolog program which explains the above instance:

```
man(Socrates).                /* fact */
mortal(X) :- man(X).          /* rule */
```

The first line can be read, "Socrates is a man." It is a base clause, which represents a simple fact.
The second line can be read, "**X is mortal if X is a man;**'' in other words, "**All men are mortal.**'' This is a clause, or rule, for determining when its input X is "mortal.'' (The symbol ":-'', sometimes called a turnstile, is pronounced "if''.)

We can test the program by asking the question:

```
?- mortal(Socrates).
```

that is, "**Is Socrates mortal?**'' (The **"?-" is the computer's prompt for a question**).

Prolog will respond "yes".

Another question we may ask is:

```
?- mortal(X).
```

That is, "Who (X) is mortal?"
Prolog will respond "X = Socrates".

Program 3:

To give you an idea, **John is Bill's and Lisa's father**. **Mary is Bill's and Lisa's mother**.

Now, if someone asks a question like "who is the father of Bill and Lisa?" or "who is the mother of Bill and Lisa?" we can teach the computer to answer these questions using logic programming.

Example in Prolog:

```
/*We're defining family tree facts*/
father(John, Bill).
father(John, Lisa).
mother(Mary, Bill).
mother(Mary, Lisa).

/*We'll ask questions to Prolog*/
?- mother(X, Bill).
X = Mary
```

Example explained:

```
father(John, Bill).
```

The above code defines that John is Bill's father.

We're asking Prolog what value of X makes this statement true? X should be Mary to make the statement true. It'll respond X = Mary

```
?- mother(X, Bill).
X = Mary
```

Program 4:

```
/* facts */
bigger(elephant, horse).
bigger(horse, monkey).
bigger(monkey, cat).
bigger(cat, ant).
bigger(lion, monkey).

/*rules*/
find(X, Z) :- (bigger(X,Y), bigger(Y,Z)) ; bigger(X,Z).
```

```
Query the following:
?- find(elephant, cat)          Answer: false
?- find(X, monkey)              Answer: X = elephant  (why X is elephant, not horse)
?- find(monkey, X)              Answer: X = ant     (why X is ant, not cat)
?- find(X, Y)                   Answer: X = elephant, Y = monkey


/*change the rule, and check the answer*/
```

```
find(X, Z) :- bigger(X,Z); (bigger(X,Y), bigger(Y,Z)).

?- find(X, monkey)              Answer: X = horse
?- find(monkey, X)              Answer: X = cat
?- find(X, Y)                   Answer: X = elephant, Y = horse
```

Program 5:

```
A binary tree Example:
branch(a,b).
branch(a,c).
branch(c,d).
branch(c,e).
path(X,X).
path(X,Y) :- branch(X,Z), path(Z,Y).      recursive call

Query the following:
?- path(a,a)          Answer: true
?- path(a,d)          Answer: true
```

Note 1:

- **PROLOG interpreter examines rules in the order in which they have been specified in the database**.
- The database created is called Knowledge Base. It is a list of facts (predicates) and rules about the domain (possible world-situation of interest).

Note 2: $x = x$

where *x* can be any *term*. Prolog *lets us express this schema* by a single Prolog fact: equality(X, X). In this fact, X is a logic *variable*. Since variables in Prolog clauses are implicitly *universally* quantified, we can read this as: "For all terms X, equality(X, X) is true."

Procedure View of Prolog:

Like normal imperative programming languages, a Prolog program consists of procedures, which are collections of statements. A program is executed by running a sequence of statements. Each of these statements executes by "calling" a procedure, which calls other procedures, etc.

Program 1:

```
fact(0,1).
fact(N,Result) :-
      N > 0,
            M is N - 1,
      fact(M,Y),
            Result is Y * N.
```

Query the following:

```
?- fact(0, 1)              Answer: true
?- fact(5, Value)          Answer: Value = 120
?- fact(3, 6)              Answer: true
?- fact(3, 5)              Answer: false
```

Program 2:

```
  sumoftwonumber(integer, integer).

  sumoftwonumber(0, 0).
  sumoftwonumber(N, R):-
```

```
N > 0,
   N1 is N - 1,
sumoftwonumber(N1, R1),
   R is R1 + N.
```

Languages that support the logic programming paradigm:

- Prolog, Absys, ALF (algebraic logic functional programming language), Alice, Ciao .....

Logic programming paradigm is often the best use when:

- If you're planning to work on projects like theorem proving, expert systems, term rewriting, type systems and automated planning.

Why should you consider learning the logic programming paradigm?

- Easy to implement the code.
- Debugging is easy.
- Since it's structured using true/false statements, we can develop the programs quickly using logic programming.
- As it's based on thinking, expression and implementation, it can be applied in non-computational programs too.
- It supports special forms of knowledge such as meta-level or higher-order knowledge as it can be altered.

Here are some online platforms that allow you to execute Prolog programs:

- **SWISH**
    - An online version of SWI-Prolog that includes plans for better shared source management, improved search for source code, and more.
    - **https://swish.swi-prolog.org/example/examples.swinb**
